POLISH-JAPANESE ACADEMY
OF INFORMATION TECHNOLOGY

The Neon Shadows : Developing the Experience of FPS Precision and Agility in Unreal Engine 5

Bartosz Mielczarek S24661
Anton Reut S24382
Antoni Szot S25074
Jarosław Fijałkowski S24468

Polish-Japanese Academy of Information Technology
Faculty: Computer Science
Department: eXtended Reality and Immersive Systems
Specialized area: eXtended Reality, Games and Immersive Systems
Thesis prepared under the supervision of:
Barbara Karpowicz, MScEng
Reviewer:
Kinga Skorupska, PhD

Warsaw, February 2025

POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

The Neon Shadows: Tworzenie doświadczenia precyzji i zwinności w grach FPS w Unreal Engine 5

Bartosz Mielczarek S24661
Anton Reut S24382
Antoni Szot S25074
Jarosław Fijałkowski S24468

Polsko-Japońska Akademia Technik Komputerowych
Wydział: Informatyka
Katedra: eXtended Reality i Systemów Immersyjnych
Zakres specjalistyczny: eXtended Reality, Gry i Systemy Immersyjne
Praca dyplomowa przygotowana pod opieką:
mgr inż. Barbara Karpowicz
Recenzent:
dr Kinga Skorupska

Warszawa, luty 2025

Abstract. The rise in demand for fast-paced first-person games has inspired the development of a new concept: The Neon Shadows, an action game about precision, speed, agility and challenging gameplay. Set in a Cyberpunk dystopia, the project focuses on the integration of responsive parkour movement and intense combat against multiple enemies. This thesis aims to describe the development process of the game, including the challenges: such as implementing procedural cutting and time manipulation mechanics, designing engaging environments in integration with parkour mechanics and maintaining such complex gameplay loop. Unreal Engine 5, offering capabilities in physics simulations, artificial intelligence, low-level programming in C++, user interfaces, animations and environment building, was a perfect fit to create such a dynamic and immersive game, while milestones, regular testing and effective teamwork helped the developers meet with tight deadlines. As a result a finished project includes precise parkour system and engaging environments, that improve freedom of movement, brutal Combat and balanced Enemy AI, that keeps the gameplay engaging, intuitive user interface and HUD, that maintains a sleek and futuristic appearance, special abilities that include time manipulation features, allowing for strategic thinking, combo points system and online leaderboards, improving replayability and encouraging speedrunning. All these systems contribute to an engaging experience, that connects ninja and samurai cultures in a unique mix and keeps players challenged, pushing them to their limits in a high-pressure environment. In conclusion, The Neon Shadows combines various systems, connects and synchronizes them in fast and complex gameplay loop. This thesis highlights the software tools, methodologies and obstacles encountered during the development period.

Keywords: Unreal Engine 5· FPP · Ninja · Cyberpunk · Parkour · C++ · Blueprints · Speedrun

Streszczenie Wzrost zapotrzebowania na dynamiczne gry z perspektywy pierwszej osoby zainspirował rozwój nowego konceptu: The Neon Shadows, gry akcji opartej na precyzji i szybkości. Osadzony w dystopijnym klimacie Cyberpunk, projekt skupia się na integracji responsywnego parkouru z intensywną walką przeciw niezliczonej ilości przeciwników. Praca ta opisuje process tworzenia gry, w tym wyzwania: takie jak implementacja proceduralnego cięcia i manipulacji czasem, projektowania angażujących środowisk współgrających z parkourem i utrzymanie na tyle złożonego gameplay loopa. Unreal Engine 5, oferujący możliwości w zakresie symulacji fizyki, sztucznej intelligencji, nizko-poziomowym programowanie w C++, interfejsu użytkownika, animowania i tworzenia środowisk, okazał się idalnym narzędziem do stworzenia tak dynamicznej i wciągającej gry. Milestony, regularne testy i efektywna współpraca pomogła dotrzymać napiętych terminów. W rezultacie gotowy projekt obejmuje precyzyjny system parkouru i interesujące poziomy, które rozwijają kreatywność poruszania się, brutalna walka i zbalansowani przeciwnicy, którzy utzymują gracza w napięciu, intuicyjny interfejs użytkownika i HUD, który zachowuje futurystyczny klimat, specjalne umiętności, w tym manipulacja czasem, które nakłaniają do strategicznego myślenia, system punktów combo i rankingi online, które zachęcają do speedrunningu. Wszystkie te systemy przyczyniają się do angażującego doświadczenia, które łączy kultury ninja i samurai w unikatowy sposób oraz stawia nowe wyzwania przed graczami, zmuszając ich do osiągnięcia swoich granic możliwości w trudnym środowisku. Podsumowując, The Neon Shadows łączy różne systemy w szybki i wieloaspektowy gameplay loop. Niniejsza praca podkreśla użyte narzędzia dewoloperskie, stosowane metodologie oraz przeszkody napotkane w procesie tworenia gry.

Keywords: Unreal Engine 5· FPP · Ninja · Cyberpunk · Parkour · C++ · Blueprints · Speedrun

Table of Contents

1 Introduction

The video game industry continues to evolve and, with the growing demand for first-person shooters, souls-like games and speedrunning community growth, the authors of this thesis have decided to create a game, that could offer all this features. A unique blend of high-speed parkour movement, speedrunning features, fast time to kill (TTK) and intense combat. The game idea is to push players to their limits, demanding precision, agility and quick thinking, while the players will push the game mechanics and level limits to the edge, an unforgiving yet rewarding experience.

When developing the first-person melee combat game, the authors decided to use Unreal Engine 5 because of its brilliant capacity for visual realization, an advanced physics engine, and Blueprints, which allow the developers to make fast prototyping and easily integrate even rather complex gameplay mechanics. Among the features of Unreal Engine 5 there are Nanite and Lumen, enabling highly detailed environments together with realistic lighting and raising the degree of dystopian Cyberpunk immersion .The engine also provides a market place that could be used not only for taking inspirations, but also integrating assets.

1.1 Goals of the work

The primary objective of this work is to design and develop the fully functional game prototype in Unreal Engine 5, called The Neon Shadows.

A very significant, yet secondary, objective is combining parkour and combat, where elegant traversing of levels feels natural while the player participates in some very intense combat moments. This involves thoughtful construction of traversal challenges that test the player's movement skills, fast and accurate combat mechanics. The interplay between these two dimensions is a core of our game and this connection should be seamless if the authors want to make a pleasant experience.

Creating an enjoyable yet challenging game loop that keeps players active, is also one of the most important goals of this project. The award is the satisfaction of accomplishment, but if the game

difficulty does not match the player skills, they won't be having fun, which is crucial. Creation of levels with growing difficulty so players master the mechanics gradually and at the same time stay highly challenged. That way, the players are rewarded with every small piece of mechanics mastered and overcoming obstacles, adding to the enjoyment as a whole.

The goal is to create a game using the latest technologies to achieve a flawless player experience.

1.2 Motivation

The authors have noticed that players demographically [20] seek titles that would test their precision, agility, and quick thinking. While most games focus either on high-speed movements or combat [17], few titles have been able to put these together and make them work in a seamlessly integrated experience. The authors are trying to fill this gap by smoothly fixing these elements together. This game is designed on the base of fast time-to-kill mechanics and agility-based challenges while travelling, hence offering a very special test of reflexes and strategic thinking by the player.

It is also a contribution to the increase in the gaming landscape for the creation of experiences challenging players to master complex mechanics. The authors strive, with The Neon Shadows, to create an intense and engaging game-one rewarding for precision, speed, and adaptability, pushing the players to reach their limits and find the full potential of the mechanics.

1.3 Inspiration

The Neon Shadows draws from a variety of inspirations that shaped the vision of the game, its mechanics and aesthetics. These inspirations are very diverse, ranging from brutal and realistic combat systems to fluid gameplay mechanics and richness of Japanese culture.

Each of these elements has been carefully integrated into the project, creating not only a unique experience, but also a fluid and logically connected mix. Atmospheric setting combined with intense

and fast gameplay. The following subsections highlight key inspirations that have influenced the development of The Neon Shadows, showcasing the creative foundation upon which the game is built.

1.3.1 Brutal and realistic combat

As the followers of movies, games, art and other forms of pop culture connected to the martial arts, cartoon-stylized depiction of brutality and comic-books the authors of the thesis took much inspiration from such artistic expressions, while creating the Katana combat for the described game. Fortunately, there is a wide range of visual and narrative material from which the authors could draw inspiration to create an authentic and engaging Katana combat experience.

The samurai movies created by famous and critically acclaimed director Akira Kurosawa or action-packed movies like the "John Wick" series showcase the characters who are driven by precision and proficiency in their actions to successfully eliminate their adversaries. The main goal is often not only the victory, but also the elegance and finesse in which it was achieved. Cinematography of such films had a great impact on the process of designing the combat of the described game, as the authors wanted to recreate the urge of thinking on the spot and improvising in very demanding scenarios during combat, making the players always revise their options carefully. Such approach discourages careless actions and provides high rewards for well executed strategy, rendering the success even more satisfying, as in the case of characters in the movies described above. While studying those examples, authors focused heavily on the tension of such encounters and the risk of imminent death to make the game described even more thrilling to experience.

The design of such games like Ghost of Tsushima (its design process was presented by Chris Zimmerman in GDC talk: Master of the Katana: Melee Combat in Ghost of Tsushima[21]), Dark Souls or even Metal Gear Revengeance had a strong influence on how combat was implemented in the game described in this thesis. Each of the games mentioned above had at least one key element that served as an inspiration for the authors while creating the experience for the players - seamless blend of offense and defense from

Ghost of Tsushima, need for strategy and high difficulty levels from Dark Souls with quick and bizarre sword play from Metal Gear Revengeance - combining all of these elements, allowed a challenging experience to be created involving combat filled with gore that realistically reacts to player's actions and has a proper impact leading to enhancement of the value of every playthrough.

1.3.2 Ghostrunner

The game authors took most inspiration from is Ghostrunner (see figure 1). What the Ghostrunner excels in is the blend of intense and unforgiving fighting with need for quickness and precision.

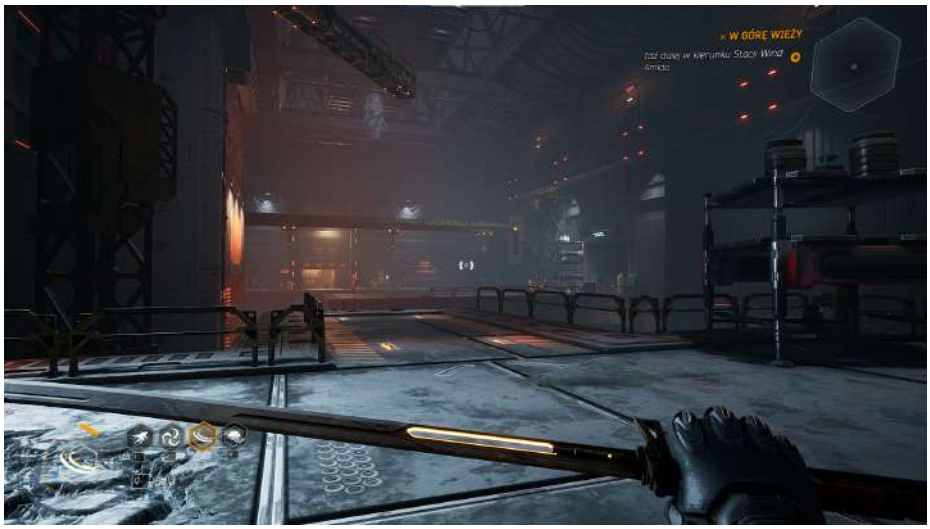

Fig. 1: Screenshot of Ghostrunner (source: own elaboration)

One of the key elements in this case is the One Hit One Kill mechanic. Intense games like Ghostrunner inspired the authors to add it ourselves. It creates high stakes where a single mistake can be deadly, thus players need to think more about their next move, which forces them to develop a plan instead of leaving the victory to a chance. Such feature demands robust combat system implementation including precise hitboxes, responsive controls and above all need for systems that aid and reward player for mastery.

The main goal of Katana Combat is creating a satisfying learning curve - something that will allow players to rethink their actions, draw conclusions from their failures and develop a devastating strategy to flawlessly beat the level. Another inspiration that was taken from Ghostrunner are the enemies. What's crucial here is their variety to ensure that player is challenged but not frustrated. Thus each of them has a unique purpose and requires different tactic that will lead to victory. For example enemy that wields a shield will require different method and pose greater threat to the player than a simple melee enemy that can be terminated very quickly. This approach heavily inspired the way combat experience is designed in the game described, resulting in interesting set pieces that can be used in interesting scenarios.

The level design and parkour is another important set piece that both Ghostrunner and the game described utilize, by mixing fast, responsive movement and parkour moves such as sliding or wall-running with increasingly harder levels allowed to create more memorable experience.

1.3.3 Japanese culture

Ninja parkour plays significant part in The Neon Shadows, defining movement, parkour mechanics and their connection with combat. Ninja culture focuses on precision, agility and the flow of movement, which is ideal for our type of game. Moving silently and swiftly, adapting to different environments has become a foundation for parkour mechanics and player experience. With every object on the map being parkourable has not only opened different traversal options, but also made the sense of mastery and skill. Players must make swift actions and master precise timing, just like in Ninja traditions. Thus Ninja have inspired us to make a gameplay experience, where players can adapt to environments and engage with enemies using map surroundings, bringing depth to the game world.

1.4 Competitor Research

The game concept finds its inspirations among successful titles that combine intense combat and fast parkour. To understand what game features resonate with the players the developers conducted a competitor research (see figure 2), all the statistics provided are gathered from Steam store [19].

1. **Ghostrunner and Ghostrunner 2**
   Ghostrunner(2020) and its sequel focuses on agility and precision in a Cyberpunk setting. These games showcase a hardcore experience that pushes player skill limits, inspiring The The Neon Shadows to include unforgiving gameplay mechanics. Ghostrunner's success is exhibited by over 1.7 million units sold with 91.6% positive reviews and total gross revenue of 33.8 million USD [3], thus demonstrating a demand for skill-based action games.
2. **Mirror's Edge Catalyst**
   Known for its parkour mechanics, Mirror's Edge serves as a model for seamless movement and immersive, fluid and diverse environments. With over 391,000 units sold and 83% percent positive reviews [8], Mirror's Edge market statistics confirms, that players value seamless movement mechanics paired with engaging level design.
3. **Neon White**
   A combination of speed and precision experience, that has created a replayable, skill-based game. It's unique mix has inspired The Neon Shadows to draw interest from the speedrunning community and adding competitive value to the game. The market success of Neon White [9], with over half a million units sold within two years, demonstrates the viability of such an approach.
4. **Dying Light**
   Combination of gore combat, fluid parkour and engaging storytelling in post-apocalyptic world was a fresh look on the zombie genre. The positive reception and popularity of Dying Light demonstrate demand for games that merge versatile movement with gore and atmospheric settings.
5. **Ghost of Tsushima**
   The impeccably designed combat, featuring precision-driven approach to sword strikes, guard braking, dodges and parries that

allows players to feel like a true samurai while providing demanding enemy encounters - it is a game design strategy that the authors of The Neon Shadows wanted to include in the game described. Upon its release in 2020, exclusively on PlayStation consoles, Ghost of Tsushima was met with both critical and commercial acclaim, praised mainly for its robust combat. The 1.8 million of units sold in just half a year is a success even for AAA project [2].

Game Title	Units Sold (mil)	Avg Playtime (hours)	Positive Reviews (%)	Revenue (mil)	Release Date
Ghostrunner	1,8	11,2	91,60%	36,3 $	Oct 27, 2020
Ghostrunner 2	0,24	6,6	82,20%	6,4 $	Oct 26, 2023
Mirror's Edge Catalyst	0,429	12,1	81,40%	4,6 $	Jun 4, 2020
Ghost of Tsushima	1,8	36	93,4 %	74,6 $	May 16, 2024
Neon White	0,488	14,1	98,30%	8,8 $	Jun 16, 2022
Dying Light	16,1	35,7	95,20%	361,1 $	Jan 27, 2015

Fig. 2: Market Statistics (source: own elaboration)

While analysing gathered statistics developers had to carefully review them considering the games difference in budget and the number of years on the market. The statistics objectively indicate that games with a strong focus on dynamic movement systems and replay ability, can not only attract a loyal player base but also achieve a sustained market success over time.

2 Game Design

The Neon Shadows is a first-person, action-packed game with elements of Japanese culture and Cyberpunk aesthetics that combines elements of parkour, intense melee combat, and traversal challenges.

2.1 Mechanics

In The Neon Shadows, the authors goal was to emphasize precision and creativity in every mechanic. The player has access to multiple traversal manoeuvres ranging from double jumps to wall runs. Such movement options improve the ability of vertical navigation and open up new paths, previously impossible to take. Combat focuses entirely on timing and skill, using Katana as a main weapon the player can attack, parry and even deflect bullets. Considering high difficulty of every encounter, the authors decided to develop special abilities that can completely change outcome of the battle such as Time Stop and Super Ability (see 5.3 and 5.6). Additionally, there are two variants of enemies which complement these mechanics, melee enemies try to shorten the distance from the player, while ranged and shielded enemies are attacking from afar, requiring flanking or usage of special abilities. In conclusion, mechanics and enemy design create a rewarding gameplay experience that encourages creative thinking and skill.

2.2 Setting

The narrative design and setting in The Neon Shadows are heavily influenced by cyberpunk, where technological integration in human body is widespread. Such alterations lead to blurred boundaries between humanity and technology raising questions about what actually can be called human. The dystopian world of this genre is known for contrasting visuals: on one side, dense, vertically layered buildings, completely covered by neon lights, and on the other, dirty streets and crumbling industrial structures populated by the poor. It becomes more than an aesthetic, an immersive landscape inviting reflection on some of the central themes in the genre and the challenges that a high-tech future may bring. The project showcases all of those elements, immersing player in the futuristic themed experience (see figure 3).

Fig. 3: Dark Cyberpunk City (source: Dominique van Velsen `https://www.artstation.com/artwork/09QWY`)

2.3 Key Factors

Due to the nature of The Neon Shadows, certain elements define its identity and set it apart from others and certain elements are mandatory in this kind of genre. These factors are critical in creating the game's core appeal. Those critical components are fundamental and must be thoroughly developed to ensure success and uniqueness of the game.

2.3.1 Fluid and Responsive Parkour Movement

Movement in the game described is an essential gameplay element that defines the player's connection to the world and surroundings, thus making movement system fluid and responsive is very important. The game should offer Ninja inspired fast-paced movement system that includes wall-running, sliding, crouching, double-jumping, mantling and dashing. All these mechanics should allow players to not only navigate through traversal levels and different environments with precision and style, but also use agility as advantage in combat to defeat enemies. Combat and Parkour as two core mechanics of the game described that are used by the player simultaneously should be connected seamlessly to a dynamic and fluid experience that players enjoy and want to replay.

2.3.2 Engaging Combat

While designing the combat experience, the authors took heavy inspiration from the games like Ghostrunner, Metal Gear Revengeance, Dying Light, Ghost of Tsushima(see GDC talk Master of the Katana: Melee Combat in Ghost of Tsushima[21]) and even from The Last of Us combat design GDc talk Unsynced: The Last of Us Melee System about melee combat [10]. The main focus was to engage players by making them think on the spot and react quickly to achieve the victory. Similarly to Ghostrunner, the game described is equipped with the one hit one kill mechanic, which pushes players agility and reflexes to the limit. One mistake and the game is over. Introducing such a mechanic was quite challenging, as the enemies must be extremely well balanced. Making them too overpowered, in order to compensate for the fact they can be killed with one sword blow, might frustrate the player. On the other hand, if they don't pose any threat, players will lose interest in playing, since the game can't provide a proper challenge. There are a couple design choices that had to be made in order to create more visceral experience. By using their Katana, players can kill enemies from much greater distance than the actual length of the blade. However this, as the testers reported, was not enough, so a quick auto-dash to the closest enemy upon every attack was added, which not only solved the problem but also made the combat encounters more fluid and dynamic.

The enemy design and variation was also carefully crafted to complement other systems. The Melee Enemy was introduced as a violent distraction. It finds the player and rushes towards him to perform a fatal knife attack. Their main role is to shift players' attention from the deadly Ranged Enemies, that fire devastating projectiles, forcing the player to move quickly and wisely. As one of the main goals is to force players to be precise, a great amount of attention was devoted to creating a Perfect Parry mechanic. Both Melee Enemy's attack and the Bullet fired by the Ranged Enemy can be parried - by performing a timed strike close to the moment when the hit lands. Using it on the former interrupts her attack while activating the slow-motion to give the player a quick moment to assess the situation, while the deflected Bullet can fly back and kill the shooter.

While designing a system that is one of the main mechanics in the game, it is not only the challenge that matters but also the impact and satisfaction derived from using such mechanic. To honour that statement, apart from the mild camera shake on sword swings and rather violent one when landing a hit, a simple gore system was added. Every single hit ends with the fountain of blood pouring from the neck and a head fling away amplifying the intensity of each kill. All key factors described above contribute to the dynamic experience and were among the crucial design elements that the described game is comprised of.

2.3.3 Speedrunning

In The Neon Shadows, Speedrunning[1] is an important factor, that could not only improve the replayability, but also attract new audience, providing another layer of satisfaction, as players strive to shave off those precious seconds after every successful run through the mastered level. The thrill of reaching the top of the leaderboard serves as a powerful motivator for those seeking self-satisfaction. The speedrunning communities even tend to create dedicated web pages to compete for the top rankings, such as Speedrun.com. Thus, the developers came to a conclusion that it is beneficial to incorporate online rankings and put emphasis on this aspect integrating it into the game. The game levels value fluidity and have multiple ways to complete each section, so speedrunning fitted perfectly, inspiring players to experiment with map layout and mechanics, while discovering efficient routes, performing chain kills and developing creative strategies.

2.4 Game lore

The lore in The Neon Shadows is the foundation that immerses players into its world. It establishes the narrative context, defines the social and technological framework. Through its cyberpunk setting and the exploration of futuristic concepts, like Neurolinks and AI,

[1] Speedrunning is the act of playing a video game, or section of a video game, with the goal of completing it as fast as possible. [source: `https://en.wikipedia.org/wiki/Speedrunning`]

the lore tells a story of rebellion, survival, and the struggle against oppression in the distant future reality.

The following sections describe the key elements of the game's lore, illustrating how each contributes to the player's understanding of the world. Although some of them are not yet implemented or shown in the game, with each level progressing through the game the player will understand the environment he is in more and more.

While some aspects are not yet fully implemented or revealed, because the developers have focused more on the technical side of the project, the narrative will later unfold them progressively with each level, allowing players to gain a deeper understanding of the environment as they advance through the game.

2.4.1 Social Framework

The game is set in a distant future, in the Cyberpunk style (see 2.2), known for its mechanical and futuristic look where humanity and technology are merged together. Governed by a regime where a privileged elite rigged elections using state-engineered meritocracy[1] to perpetrate its rule. The setting of neon lit circuits the so called progress of huge skyscrapers above the shantytowns indicating the horrifying contrast of those that rule and those that are enslaved.

The elements of Science-fiction are inclusive throughout this world, starting with Neurolink and ending with AIs that control the world. Beneath and within this showcase of the most advanced technology capturing individual freedom, there is a feeling of hopelessness. The world is of radical insurgency and of existence in which, any act of defiance brings the player closer to the reality.

2.4.2 The Neurolink and Control

The Neurolink is a modern concept that could not only help disabled people, but increase human brain capabilities, improve efficiency. These technology is used by the main character to his advantage, providing information displaying, obstacle analysing and risk

[1] Meritocracy - a government or the holding of power by people selected according to merit.

calculation. Those elements are integrated into the in-game User Interface and Head-Up Display.

But Neurolinks as a concept have a few downsides and concerns, such as privacy and safety. Our lore explores those concerns and manufactures them into the new way to control the society without peoples knowing, the control of though that can be a very powerful weapon. This shows how certain inventions can be manufactured into weapons of mass destruction, like TNT or Nuclear Reactors.

2.4.3 Artificial Intelligence

Artificial intelligence in the future can become not only a great tool for automating common tasks, but also something, that can replace human workers everywhere or something, that can be used as a weapon. Combining such a powerful structure with Neurolinks, the rule has created a powerful system that monitors most of the citizens. Using Neurolink AI is influencing emotions and suggesting thoughts, thus controlling peoples behaviour. The AI categorizes “undesirable” thoughts and clears “bad” minds by either reprogramming them or eliminating individuals entirely, under the guise of maintaining order and societal advancement. This aspect also is influenced by controversy around the Artificial Intelligence and its impacts on Humanity.

2.4.4 Kitsune Symbolism

Fig. 4: Kitsune Mask (source: `https://www.ebay.pl/itm/165147791423`)

In Japanese folklore, Kitsune are foxes that possess paranormal abilities that increase as they get older and wiser. They are seen as supernatural beings, stealthy, able to transform, wise and beautiful. While some folktales speak of kitsune employing their abilities to trick others, as foxes in folklore often do, other stories portray them as faithful guardians, friends, and lovers. In Japanese culture Kitsune masks (figure 4) are used a lot during Rice Harvesting festivals and Celebrations.

The main character starts carrying around the Kitsune mask, because it is the only thing she has left from her parents, thus the only thing she can be sure she will remember, the only thing the AI cannot alter in her memories. It becomes a reminder of personal revenge and redemption.

The main character also embodies the Kitsune characteristics, at first being careful, agile, stealthy and tricking others, but later being an open-hearted guardian for the citizens, becoming wiser.

The Citizens seeing such a fighter were inspired to resist against the system too, because it took something from everybody. Thus, throughout the game, this Kitsune mask became a symbol of the whole movement, because it was a very distinctive part of the character. This emblem resonates deeply with rebels, embodying their desire to escape manipulation and think freely. Kitsune graffiti and symbols mark hidden meeting spots and safehouses within the city, helping to unify resistance fighters and inspire others. Their goal is to evade the ever-watching AI, and ultimately destroy the Neurolink government control.

2.4.5 The Mission

Although in The Neon Shadows the player's goal is to overcome the harshest conditions and different obstacles, the ultimate story goal is freedom. The player's mission is to disable the AI control, knowing it will end the elite's power grip and give citizens back their freedom of speech. It's a struggle with danger and sacrifice, where every step taken toward disabling the AI is a step closer to liberating society from its own creation.

3 Related Works

3.1 Math in Games

The knowledge of vector and matrix operations, coordinate systems, quaternions or any other mathematical structure and its applications should be obligatory for every game developer. The book titled *3D Math Primer for Graphics and Game Development* is an excellent introduction to the mathematical underpinnings that many 3D games are built upon, and served as both a guide and an inspiration throughout the development of the project described [13].

3.2 Procedural Animation in Game Development

The Game Developers Conference (GDC) talk titled *Procedural Animation in Game Development* presents a robust and a very unique approach to procedural animation generation, as the animations themselves, are usually comprised of just one or two frames (serving as a reference) and the rest is generated by a creative interpolation and mathematics. High quality of the talk, its innovative ideas and the visually pleasing results of the presented methods inspired authors to implement their own procedural animation solution. While not as advanced as the topics mentioned in the talk, the results of the authors work did rise the quality of the visual side of the game described to some extent [18].

3.2.1 Game Programming Patterns

The book by Robert Nystrom describes a number of regular programming patterns and puts them in an appropriate game programming context, while highlighting both the advantages and disadvantages. Some of those were used extensively throughout the development of the project, including the Observer pattern and Singleton pattern [11].

The programming rules presented by Chris Zimmerman - a cofounder of Sucker Punch Productions - can be used to write maintainable code. The following book was chosen because of the author - Chris Zimmerman is long-time game industry veteran who

co-founded the Sucker Punch Productions and published the game Ghost of Tsushima that had an influence on the design of the game described. Authors believed that his expertise in the field of game programming can be put to a good use while creating their project [22].

The Options pattern, used also in .NET programming, allowed the authors to customize the behaviour of given functions with a different method of passing the parameter list while simultaneously increasing the code readability and scalability [12].

3.3 Combat System Design in Games

In the field of the design, another work by Chris Zimmerman (*Master of the Katana: Melee Combat in Ghost of Tsushima*) guided the authors through the development of the game described. The following GDC presentation follows the steps Sucker Punch had taken while creating the combat loop of Ghost of Tsushima. It is full of advices, rules and guidelines that benefited the design process of the game described [21].

The *Unsynced: The Last of Us Melee System* GDC presentation is also worth mentioning, since it refers to the melee combat. Its main topic is the feel, believability and impact of the hand-to-hand combat. Furthermore, it gives guidelines how to create sync takedowns, which does give the authors another area the game described can be improved upon [10].

3.4 Parkour in First Person Games

The *Parkour: How to Improve Freedom of Movement in First-Person Games in 20 Simple Steps* GDC lecture is about how the parkour was created in Dying Light. It was presented by Techland's Bartosz Kulon, that shared examples of problems and how they solved them during the development process. This information gave the developers an understanding of parkour basics and how to implement them. Besides this, lecture also describes jump assist that was partially implemented in the project and gives a lot of advices on how to make movement smooth, achieve fluidity and avoid motion sickness [7].

4 Tools and methods

This section describes the tools and method, that were used during the development process of The Neon Shadows.

4.1 Unreal Engine 5

Unreal Engine 5 served as the primary development tool for The Neon Shadows, offering a very powerful toolset that allows:

- changing the engine code and coding in C++
- visual scripting (such as Blueprints)
- creating and editing animations, animation blueprints, meshes and maps
- developing SoundFX, PostFX and cinematic sequences from scratch
- designing User Interface and Heads Up Display elements, animations and interactive screens

Unreal Engine 5's massive toolset, flexibility and novelty made it the ideal choice for developing a game as complex and visually rich as The Neon Shadows.

4.1.1 Blueprints

Blueprints in The Neon Shadows were often used, because of their flexibility. They are able to quickly and visually code logic, that integrates complicated functions and simplifies workflow for the whole team.

Not only do the blueprints help to maintain the code and the actions in the game, but they are also easy to understand due to their visualization. This makes it easy for any team member to comprehend all parts of the project, if they were not directly involved in its implementation.

In addition, the already implemented class-based system allows integrating project parts in C++, which is great for the project as The Neon Shadows utilizes the more powerful C++ in lots of its main game mechanics.

4.1.2 AI Assets

The whole concept of AI programming is already implemented in Unreal Engine. Its complexity is simplified, which enables the project to include more advanced systems. The artificial intelligence system is mainly presented in four assets:

1. **Behavior Trees**
   Behavior Trees are the main asset in the AI programming. Their branches show how the process of flow should go. It is a good representation of what is happening within the enemy during gameplay. Behavior Trees consist of:
   - **Behavior Tree Tasks**
     Behavior Tree Tasks are the single actions that AI do. In Behavior Trees, they are represented as nodes. These include custom ones, such as "Wait," and new ones created by authors, which is more common. In The Neon Shadows project, most of the Behavior Tree tasks are created from scratch, for instance: BTT_ChasingPlayer or BTT_AttackPlayer.
   - **Behavior Tree Decorator**
     They are used as simple conditions within the Behavior Tree. There are already custom ones like Blackboard decorators, that are used constantly in The Neon Shadows. Moreover, you can create ones. In The Neon Shadows project's case the best example is BTD_IsWithinIdealRange, that checks whether the player is in the best suited place for the certain action to perform. Due to its complicated performance, the unique Behavior Tree Decorator was necessary .
   - **Services**
     Services are nodes that have their own frequency of action in the Behavior Tree. Their parallel execution works perfectly to calculate additional parameters that are essential to monitor. In the case of the The Neon Shadows game, one service is responsible for calculating distances between the enemy and the player. It is highly important to do this constantly, and in the graph, it fits perfectly.
2. **The Blackboard**
   For each Behavior Tree, a blackboard is needed. It contains vari-

ables that control the Behavior Tree's entire procedure. The system of Blackboard decorators is based on their existence in the project. They can be used in any other node at any time.

3. **AIController**
   Their role is based on the most primary decision in AI mechanics. They decide on actions such as whether the AI should run or not. Moreover, they control the connection between all AI assets and the main enemy's blueprint.
4. **Environment Query**
   The Environment Queries are an advanced sensing system that allows specifying exactly how calculating every Blueprint's position in the scene should be considered. They are able to create a simple grid on the ground and specifically maneuver each position's object within it. They are used as additional help for the enemy to interact with the environment and the player, making the project more realistic. This system helps improve the AI recognition of surroundings and its interactions with them.

4.1.3 VCS Merges

Version Control System (VCS) is a practice for managing, organizing, and tracking different versions of computer files. For the game development projects, use of a VCS is crucial due to the scale of the projects, the iterative nature of their development, and the need of collaboration among team members.

For this project, the developers have chosen GitHub [5] as their Version Control System, based on this solution cloud-based infrastructure and the team experience with this platform. However, a significant limitation of GitHub is that it does not support to merge Binary files, that posed a challenge for the team. Unreal Engine stores most of its assets (including Blueprints, Materials, and AI Assets) in binary format, making this limitation problematic.

An alternative solution - Perforce [14], offers binary file merging capabilities. However, it requires a local server installation, which was not an option for the developers at the time.

So, to address this issue of merge conflicts, the team members maintained consistent communication regarding the systems they were working on. When the conflicts were unavoidable, then the two

(or more) copies of the file were either merged together manually in the engine or only the one version of the file was selected to proceed with. Regular merges (every week) have also helped to minimize conflicts and maintain stable development workflow.

For merging purposes the authors have primarily relied on Rider [16] capabilities. This software allowed for automatic merging, while still providing manual controls to handle more complex merge conflicts. This made it an effective solution for managing the GitHub repository in the context of Unreal Engine development.

4.1.4 Control Rig

Unreal Engine's Control Rig is a specialized Blueprint used specifically for procedural animation programming. Unlike the previous solutions inside the Animation Blueprint, Control Rig is a stand-alone asset that contains a dedicated graph allowing for the basic flow control, use of data structures and custom functions along with dedicated procedural animation Nodes. Control Rig is the right tool for a wide gamut of potential applications, including Inverse Kinematics, Rigging or Dynamic Hierarchy management that allows solving more complicated and demanding problems than in previous iterations. It was a crucial tool for implementing the procedural animation system described later.

4.1.5 Plugins

Plugins are modular components that extend Unreal Engine's functionality. They provide additional tools or features which allowed the authors to expand the functionality of the project efficiently. The Neon Shadows utilizes the following plugins:

1. **Did It Hit**
   DidItHit plugin is a simple solution for a collision tracing in a specified layout, for instance around the blade of a sword. The options allow for a number of different types of traces - with a specific shape, by specific channel, or for specific objects. It works by creating the selected collision trace between specific Mesh Sockets and storing the hit data in a dedicated array - a

feature that was leveraged by the authors while implementing the Katana combat system.

2. **Advanced Sessions**
   The Advanced Sessions plugin is most commonly used for creation and managament of multiplayer sessions, especially on Steam platform. However, in The Neon Shadows its usage is only limited to Steam profile integration used by save system and leaderboards, since Steam ID and Username are needed for them to function with all present features.
3. **Epic Leaderboard**
   The Epic Leaderboard plugin is a tool used for integrating online leaderboards into any Unreal Engine project. The use of that plugin was based on its simplicity and quick implementation without relying on Steam, where such a feature is behind a paywall.

4.2 Content

The content creation process for The Neon Shadows involved developing various elements, including 3D models, level design, UI components, and other visual assets. The following tools and methods were used during the development process:

- **Blender**
  Blender is a free and open-source 3D computer graphics software tool that is used for creating animated films, visual effects, art, 3D-printed models and more.
  In this project it was utilised in modifying 3D models when Unreal Engine's built-in tools were insufficient.
- **Adobe Illustrator**
  Adobe Illustrator is a vector graphics editor and design software developed and marketed by Adobe.
  It was used in creating and developing the User Interface (see 5.9) elements, due to its precision and scalability for vector-based assets.
- **Adobe Premiere Pro**
  Adobe Premiere Pro is a timeline-based non-linear video editing application developed by Adobe.
  It played a key role in creating milestone functionality showcases and trailers for The Neon Shadows.

4.3 Time management

Due to the author's circumstances, such as studying and working beside the project, every team member had their own work schedule, so to be flexible, yet efficient and adaptive the authors have chosen Agile methodology [2] to manage this project.

The authors have adapted Scrum [3] to their circumstances, creating an efficient workflow that helped the developers with tight deadlines, regularly scheduled tests and milestones.

The developers have built a Game Design Document (GDD) as a start to create a common picture of the game between all the team members. Later, the GDD served as a feature plan and a documentation of the project.

Using Jira as a sprint planning tool [4] the authors have created a Backlog and a Kanban of tasks providing themselves with planned game mechanics the team needs to implement. Using 1 and sometimes 2 week sprints with at least 2 meetings per week they were adaptive, dividing responsibilities and providing more resources to the more important points of the gameplay. Finishing each Milestone (3-4 weeks each) the developers team has concluded user tests with additional test protocols for each newly implemented feature. After each concluded test they had a "Test Results" document with new ideas on the gameplay, found bugs and overall test observations, that helped later improve Level layouts and new features, providing more information on where to put more resources, to make the game a complete experience.

4.4 Programming techniques

Along with every new feature added, the overall complexity of the project increased. To ensure concise and flexible architecture that is open for modification and decrease coupling between systems - apart from the programming rules gathered in Chris Zimmerman's

[2] The Agile methodology - is a project management approach that involves breaking the project into phases and emphasizes continuous collaboration and improvement.

[3] Scrum - is an agile project management framework that helps teams structure and manage their work through a set of values, principles, and practices.

[4] Jira - is a proprietary product developed by Atlassian that allows bug tracking, issue tracking and agile project management (https://www.atlassian.com/software/jira).

book [22] - a couple of Design Patterns and Data Structures were employed.
They include:

1. **Observer Pattern** - the pattern(described also in Game Programming Patterns[11]) where one entity acts as an observer and the other is a subject of that observation. Once the subject performs an action, it notifies all of its observers, so that they can react accordingly. This pattern was used extensively while writing the communication between the code responsible for the gameplay mechanics and its User Interface counterpart. In Unreal Engine this pattern is implemented in the engine in a form of delegates(for sending) and events(for receiving).
2. **Singleton Pattern** - the pattern (described also in Game Programming Patterns[11]) that ensures that only one instance of the class can exist in the entire program. If an attempt is made to create a new instance, it will return the same instance that was created earlier. It is especially useful when there is a need to have only one instance of a class globally, for example Save Data Managers or in objects that need to persist across multiple levels.
3. **Options Pattern** - also known as the **Parameter Object Pattern** involves passing a struct with the list of parameters into a functions instead of declaring those parameters individually in the function signature. [12] When using this approach, not only does the function calls with many parameters look cleaner, but also values for the default parameters can be specified in a random order.
   This pattern was used in the Combat System to customize the target validation algorithm, depending on the context of use.
4. **Object Pooling** - the pattern that minimizes CPU load by reusing objects instead of constantly creating and destroying them. [11] A predefined number of GameObjects are created before the game starts. These GameObjects are then activated or deactivated as needed and recycled back into the object pool - a storage location for reusable objects. This removes the need to instantiate new objects or destroy existing ones during gameplay. In The Neon Shadows, this approach is used for bullets fired by ranged enemies.

5. **State Machine pattern** - is a behavioural design pattern (described in Game Programming Patterns[11]) used to manage an object's behaviour based on its current state. This pattern consists of three main components: states, transitions and encapsulations. Using this pattern the Animation Blueprint is created, where each state and transition has its own animation, creating fluid animation handling.
   This pattern ensures a clear separation, by dividing state-specific logic and simplifies handling complex systems with many states.
6. **Ring Buffer** - also known as Circular Buffer, is a fixed-size buffer that is circular in nature. When memory is generated and consumed, data do not need to be reshuffled, only head/tail pointers are adjusted. When data is added, the head pointer advances. Similarly, when data is consumed, the tail pointer advances. If the end of the buffer is reached, the pointers simply wrap around to the beginning. [6]

5 Results

The main goal of the following project was to develop the experience of FPS Precision and Agility.

By using Unreal Engine 5 the authors have developed a sophisticated experience that involves multiple systems, described below. All of these systems contribute to the fast and complex gameloop, fulfilling the goal stated in the thesis.

5.1 Parkour system

The movement in The Neon Shadows is a core gameplay mechanic, thus making it responsive and fluid is crucial to achieve an engaging player experience and navigation through different environments. It should give the player a feeling of a ninja-like character - precise, fast and agile. Yet if the movement becomes too fast, it would be hard for the players to control the character and to play the game in long sessions. To achieve this balance, the authors chose to combine Unreal Engine Physics with advanced mathematical calculations.

The project's movement system is a combination of Animation blueprint and Unreal Engine's character movement with a custom Parkour Movement Component, that manages all the added parkour mechanics described below and overrides default movement methods in different parkour states. The main Character blueprint (Third person Character blueprint) is extended by this Parkour Movement component. Event mechanics are triggered from the Character Blueprint using the Enhanced Input System. While timelines are defined within the main Character Blueprint, the rest of the system's functionality is exclusively managed within the Parkour Movement component.

The Component's structure is divided into sections:

1. **Initialise** - where all the needed variables, that keep track of the default parameters, are initialized.
2. **Movement change trigger** - that keeps Current and Previous movement variables updated.

3. **Update event**(see figure 5) - that is set to repeat every 0.0167ms = 1/60s (1 frame in 60fps) to update the character movement in accordance to current parkour movement mode.
4. **Land event** - that is used to reset movement and some variables.
5. **Jump event** - that is used to override default jump mechanics if needed.
6. **Wallrun** - mechanic that allows player to run on walls.
7. **Crouch/Slide** - mechanic that allows player to slide under the obstacles.
8. **Mantle** - mechanic that allows player to climb up edges.
9. **Dash** - mechanic that allows player swiftly move in a straight line.
10. **Queue** - that remembers the last input that needs to performed after landing (for example).

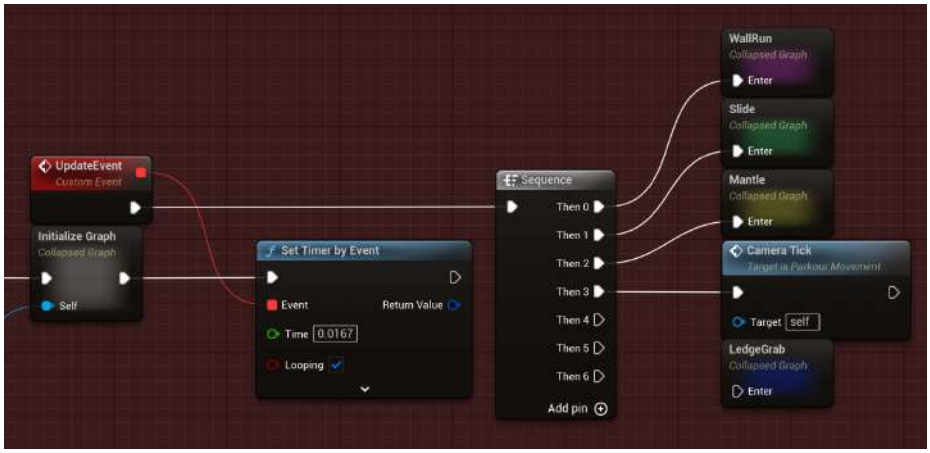

Fig. 5: Update event that on Tick calls every Parkour mechanic that requires an Update and a Camera Tick (source: own elaboration)

The Update event (see figure 5) serves as the core of the Parkour movement component. While some mechanics are triggered by player input, others, such as Wallrunning or Sliding, fundamentally change the player's movement. To maintain these dynamic behaviours, the Update Tick must execute them at least 60 times a second. The Update Tick operates as a sequencer, checking each movement's Gate.

If a Gate is open, the corresponding movement's update method is executed. A Gate is a Blueprint node that includes an Entrance and an Exit, with customizable Open and Close methods or events to control its availability. This approach is more optimized compared to using a standard Branch with a Boolean, as it enables the creation of complex logic while maintaining a structured and efficient design.

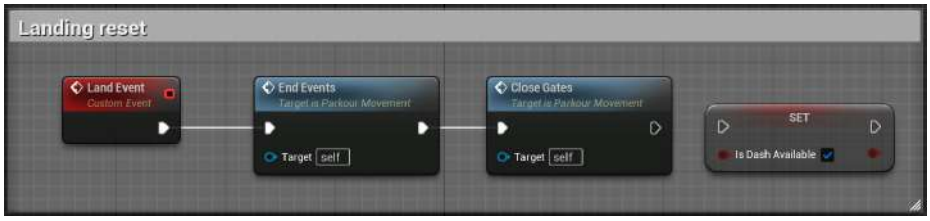

Fig. 6: Land event in Blueprints (source: own elaboration)

The Land Event (see figure 6) is triggered when the falling player character makes contact with the ground and transitions into the walking state. This event serves a critical role in the Parkour system, acting as a signal to reset Parkour mechanics. Specifically, it closes the parkour state's gates and terminates any active Parkour mechanics such as wallrunning, sliding, and mantling. Upon landing, the player enters Unreal Engine's default walking mode, where none of the Parkour movement modes are defining the character's movement. The Jump Event is triggered when the player performs a jump action. To support varied jump behaviors depending on the current Parkour state, this event is overridden within the Parkour component. Upon triggering the Jump Event, the Parkour component first evaluates whether the current Parkour movement mode is associated with some Jump event. If custom Jump event is identified, the component ends the active Parkour mode and executes the custom jump. If no custom behavior is applicable, the system defaults to performing a regular Unreal Engine jump. By customizing the Jump Event, the Parkour system ensures that jump mechanics are adapted to the player's current context, maintaining fluidity and responsiveness within the Parkour movement system.

The Queue in the Parkour system is designed to store actions temporarily and execute them later in the correct order. In the current version of the Parkour system, the Queue is primarily used to enable a Slide action while the player is in the air and falling. Some of the actions that are not available to be performed are remembered in the Queue for a duration of 0.5 seconds. If the player triggers a Land Event within this time frame, the queued actions will be executed immediately upon landing. As a result, the player transitions into the Slide action after landing, maintaining fluidity in gameplay.

5.1.1 Movement and Camera

The regular Unreal movement was modified in order to create a more balanced, quick and responsive player feeling. The authors have increased the character's air control, increasing air brake force and acceleration. The friction with the ground was also increased to make the player ground movement more precise. The character mass and capsule have been decreased and the jump force was slightly increased. All these changes made the gameplay faster and more skill dependent, while the character started to feel more like a light ninja, that is capable of controlling their body like it is an agile weapon of mass destruction. The most important thing for the authors was not to overdo the movement speed, because on the experience and statistics of other games in the genre, the playtime sessions are pretty short. For example, in Ghostrunner, some user reviews (provided by the Steam Marketplace [4]) say that their eyes and brain are tired of playing. To avoid this issue, the authors have tried to avoid making movement too fast and decrease its influence on users health.

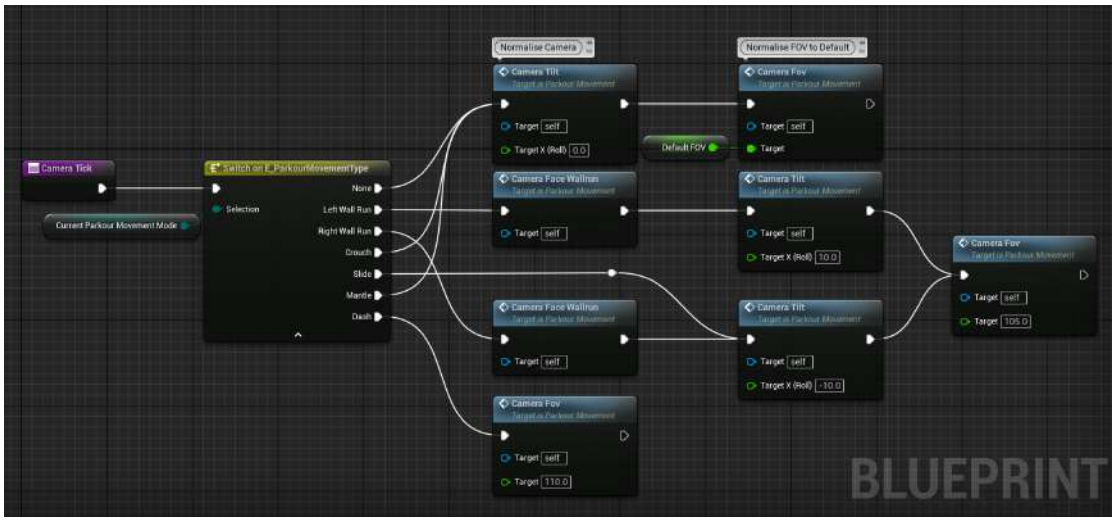

Fig. 7: Camera tick (source: own elaboration)

To increase the feeling of speed and character responsiveness the authors focused on refining the first person camera to make an immersive impression of motion. The camera's update cycle (see figure 7) is connected to the Update tick (see 5.1 Update tick), so it has the same 0.0167 second operating frequency (1/60 of a second). During each tick, camera parameters are dynamically adjusted depending on the current Parkour mode, using linear interpolation to transition smoothly from default value to mode-specific settings. The primary camera adjustment in Parkour system include changes of the camera are Field of View (FOV) and camera tilt, that does not only amplify the perception of speed and immersion but also provides an increase of the players control. The camera tilt serves as visual cue indicating the active Parkour mode, thereby improving players situational awareness. Additionally, the integration of motion and radial blur further immerses players in the fast-paced gameplay, reinforcing the adrenaline nature of the game.

5.1.2 Double jump

As the game lacked movement capabilities when it comes to vertical traversal, the authors decided to develop Double Jump mechanic.

The goal was to open up more paths for the player, encouraging creativity in platforming and combat.

The first jump was implemented by using already existing Jump function from Unreal Engine, which is simple, but does not give much control. However, the second jump uses LaunchCharacter function that allows for easy force application to the player character. Force is applied based on player input, meaning it is possible to perform it in any direction on the X and Y plane. However, on the Z-axis, the force is constant for every jump.

It is also important to note that in a complex parkour system, the double jump needs to be seamlessly integrated with other game elements such as wallrun. A boolean variable that activates at the end of the wallrun allows the player to perform a second jump after bouncing off the wall. The number of double jumps is unlimited, provided that the player touches the ground after performing each double jump. This approach is restrictive enough to challenge the user, but at the same time rewarding enough to flawlessly introduce new strategies in beating levels.

5.1.3 Dash

The Dash is the mechanic that allows players to travel swiftly in line in the direction they want. By using this mechanic, players can gain advantage on the enemies by quickly repositioning themselves or use it to overcome parkour obstacles in combination with Double Jump (see 5.1.2).

The first version of the dash mechanic was just a regular Launch Character, that was throwing a player in the direction of his inputs, but this solution lacked precision, it felt like a trampoline bounce. Tests have shown that it was used mostly by players in parkour to overcome obstacles and give themselves a boost forward in air, while its usage in combat was too unpredictable, so most of the players have avoided it.

The authors have decided to provide more control to the dash mechanic and make more predictable. In order to do so, they have made a decision to make the Dash mechanic with more of a Mathematical approach, thus the Dash mechanic was implemented using linear interpolation.

The Dash event of the Parkour movement system is called from the main character blueprint using Enhanced Input system.

The Dash implementation can be divided into 3 main components:

1. **Dash opening** - that checks if the dash is not colliding with other parkour mechanics and checks the dash cooldown.

2. **Dash parameters calculation** - where all the parameters used in Dash are set depending on Lerp wallhit and Dash direction

3. **Dash Timeline** - that updates character's position using sweep Lerp and at the end resets current parkour mode and gives the character a end velocity.

The Dash opening performs checks, if the mechanic is not colliding with other parkour mechanics, then checks for the cooldown delay.

The Dash parameters calculation first starts with getting the player stats like position, velocity and last input to make a trace (from character position into the last input side multiplied by dash distance variable, see 8). If the the trace hits an object the destination point of the dash should be trace hit length minus player capsule size, to avoid players getting stuck into the wall. Additionally after shortening the dash distance, the dash timeline play rate should be fasten equally to avoid inequality of performed actions speed and make it even more precise.

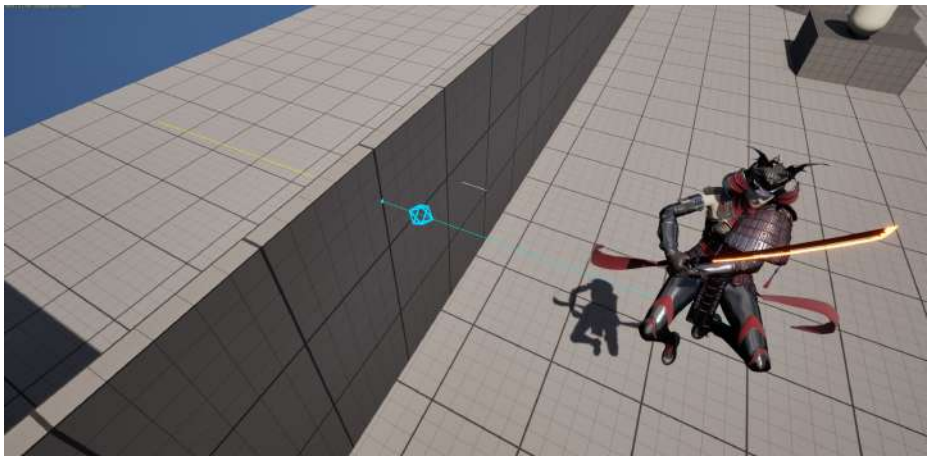

Fig. 8: Cyan trace is before hit, yellow is after. The Cyan capsule shows the Destination point (source: own elaboration)

The timeline has an Alpha parameter, that increases from 0 to 1 using a prepared curvature (formula). On the screenshot of the timeline (see 9) the curve incline to the value of 0.20 is steep on the beginning, because usually when performing dash the character already has starting speed, thus players can still keep the feeling of continuity. This value in timeline update method goes into the Lerp method, that giving start position, end position and alpha, returns an interpolated position of where player should be. At the timeline end, the player's velocity is deliberately set to a value before dash in sum with dash force parameter that differs from the direction of the dash. This is to prevent the velocity from defaulting to zero after the interpolation and ensures smooth movement continuity and momentum saving. For instance, a forward dash is assigned a higher velocity due to its nature, reflecting greater momentum compared to dashes performed in other directions. Sweep option of the player's position changing method ensures the dash will not force character into the wall.

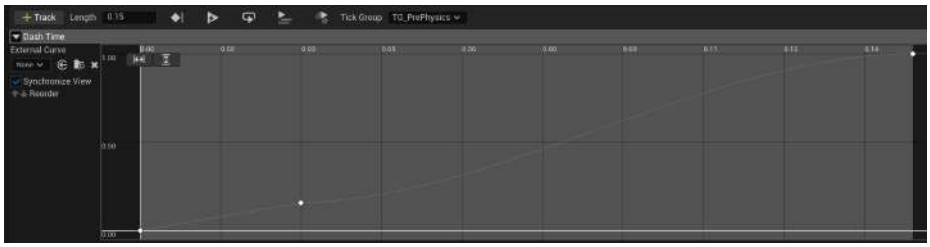

Fig. 9: The Dash timeline curve (source: own elaboration)

As a result the Dash mechanic turned out to be very popular among the players. Allowing the player not only to move fast and jump further, but to reposition themselves against an enemy combatants, thus gaining an advantage on the enemy.

5.1.4 Mantle and Ledge Grab

The mantle mechanic is a movement ability that allows a character to climb over or pull themselves up onto an obstacle, such as a ledge, wall, or other elevated surfaces. The ledge grab is a mechanic that allows the player to hang on the wall, while being able to look around before getting up the ledge. Conducting the tests, the developers have noticed that players often faced frustration when narrowly missing a platform by just a few centimetres, resulting in their character falling to their death without the ability to climb onto a ledge. This issue proved particularly annoying in a fast-paced and restrictive gameplay environment, as it disrupted the flow and penalized players for minor missteps. To avoid frustration and improve player mobility making environments more interactive and realistic the authors have created a mantle ability that in first version had two modes: Ledge Grab and Quick Mantle (only the second one made it through).

This ability can be divided into the following sections:

1. **Mantle Check Update** - function that check if the Mantle can be performed
2. **Quick Mantle Update** - performing a quick mantle
3. **Ledge grab** - sticking to the ledge
4. **Ledge grab mantle** - performing ledge mantle

Mantle Check starts with its own gate that opens when the character is falling and has no other parkour modes active. If the gate is open, then mantle check is being performed by making a capsule trace from character eye level to the feet (see figure 10), but both the points are shifted in front of the character to the distance of its capsule radius. If the trace has hit something and by the hit distance, the developers can ensure, if it was lower than the character's waist. If it was higher, then the authors should perform the Quick Mantle, if not the Ledge Grab should be performed. Before setting the Parkour mode to Quick Mantle or Ledge grab and opening the proper gate, the calculations of the start and end points of the mantle should be performed.

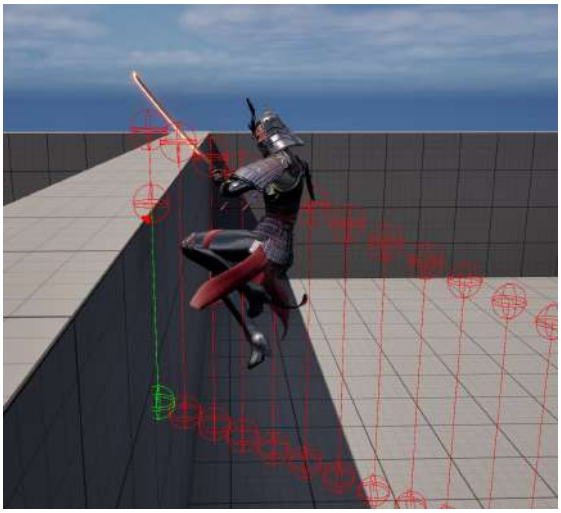

Fig. 10: The Mantle Capsule trace. Green line is after the trace hit, red line is before the trace hit (source: own elaboration)

The Quick Mantle Update translates the character position swiftly using Timeline. The Ledge Grab stops the character movement completely, leaving him only the availability to look around and then if the forward input is not 0, the Mantle can be performed during Ledge Grab with different timeline play rate speed and position changing. Conducting another set of tests the developers have discovered that the Ledge grab was ruining the feel of the rapid gameplay and con-

tinuous parkour, where the quick mantle was very appropriate and not interruptive.

In conclusion, the mantle mechanic has been integrated into the parkour system. Although its usage often goes unnoticed by players, it is a frequently used feature, which is the effect the developers were trying to achieve, reflecting a natural and intuitive experience.

5.1.5 Wallrun

The Wallrun is a mechanic that allows a player to run on the wall. This action was also popular among ninja as it allowed them to overcome different obstacles. The developers have made a decision to only implement this feature, but to make every object on the map wallrunable and the wallrun to be infinite, so the player will not fall while performing it.

Before enabling Wallrun movement the Parkour system should determine if the Wallrun can be performed.
First criteria is the character Falling, this way the system can determine, that the character is in air.
Second is two line traces at each side of the character (left and right) for the Wallrun check. The traces start is an actor position, and the end vector ($\vec{v}$ is a sum product of character position ($\vec{p}$), actor right vector ($\vec{r}$), coefficient (k) that is used to invert the right vector (right side equals 1 and left side equals -1) and actor front vector ($\vec{f}$).

$$\vec{v} = \vec{p} + (\vec{r} * 65 * k) + (\vec{f} * 25)$$

Both of the traces (see figure 11) are brought in front of the player by the multipliers, because instinctively most of the players face the wall they are jumping on. If one of them hits an object, then it means that the player is close enough to the wall to perform a wallrun.

Fig. 11: Red Wallrun Check traces that determine if the wallrun can be performed (source: own elaboration)

Starting a wallrun player characteristics are changed: the gravity is set to 0, so the player would stick to the wall vertically, the player movement speed is increased in favor of game balance and increasing the feature usage, the target camera FOV is increased to 105, the camera is being tilted by 10 degrees, depending on the wallrun side.

To make wallrunning more intuitive, the camera rotation is smoothly interpolated in the direction of the wallrun, by using the wallrun direction vector, which is obtained by calculating the cross product between the wall normal and the up vector. Its orientation can be used as the target rotation as seen in figure 12.

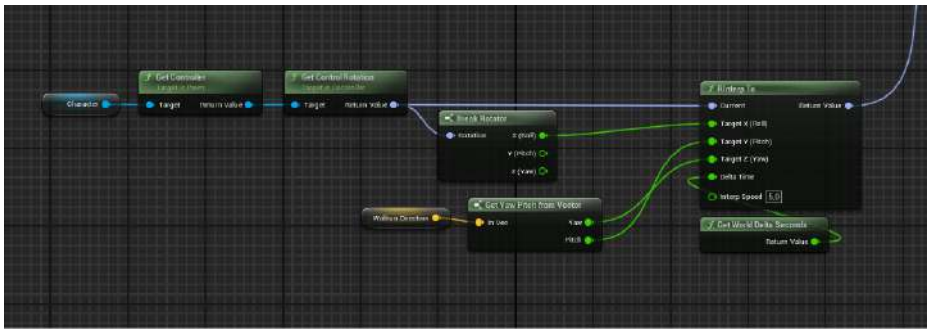

Fig. 12: Wallrun camera rotation calculation (source: own elaboration)

As the main point of such system is to assist the player, the cancellation rule was introduced presented in figure 13. It disables the camera rotation system if the player's per-frame mouse movement was rapid enough.

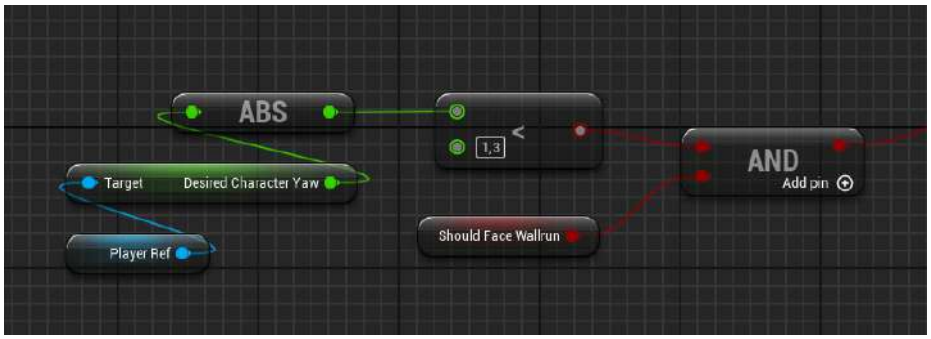

Fig. 13: Rule for entering the directional camera update (source: own elaboration)

After the wallrun has started the "Wallrun movement" checks if the player is still running on the wall using two side rays, their start position is the same as previously (character position), but the target position is slightly pointing backwards and the trace distance is increased, so the player is able to look around while performing a wallrun action. The end vector ($\vec{v}$) of the wallrun update trace is calculated using the sum product of actor position ($\vec{p}$), inverted

forward vector ($\vec{f}$) and right vector ($\vec{r}$), where coefficient (k) depends on the side the trace is on.

$$\vec{v} = \vec{p} + (\vec{r} * 120 * k) + (-15 * \vec{f})$$

In conclusion, the wallrun mechanic did not only open players to a

Fig. 14: Wallrun traces that determine if the wallrun can be per-formed (source: own elaboration)

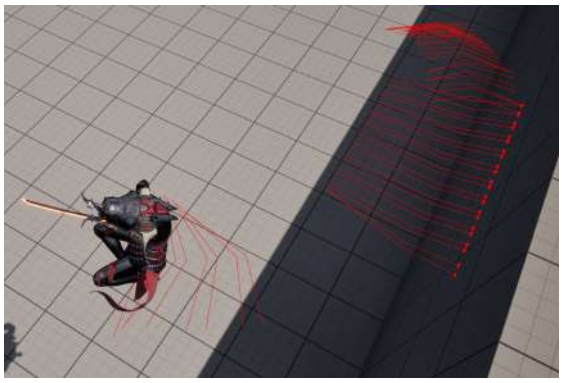

Fig. 15: Wallrun traces transferring from Wallrun Check to Wallrun Update and backwards (source: own elaboration)

whole new perspective of overcoming obstacles and enemies, but has immersed them deeper into the ninja culture and game lore.

5.1.6 Crouch/Slide

Crouching allows players to reduce their height, allowing them to dodge damaging enemies or projectiles, enter tight spaces or make themselves more stealthy. Crouching while running will make a character perform a ground slide in the same direction. During sliding the character's course cannot be changed and the actor velocity is dramatically increased when sliding down towards the floor slope. In addition, crouching can be used to cancel various parkour states, for example, Wallrunning or Mantling.

The Crouch/Slide mechanic in The Neon Shadows is a combination of Unreal's crouch movement, physics manipulations and pa-

rameter altering. The Crouch event is triggered by the Enhanced Input system from the Blueprint Thirdperson character.

First the Crouch event checks the current Parkour state, if it corresponds to a state that could be cancelled, then the corresponding mechanic End method is triggered (for example Wallrun End 5.1.5). If the current mode is Crouch or Slide, then the Uncrouch method is called. The Uncrouch method performs a capsule trace using players height and, if the capsule trace did not hit any meshes (static or dynamic), then the player's capsule is raised and walking parameters and animation are reset. The Crouch method, on the other hand, decreases the player's capsule, changes its physics parameters and changes actor state in the character animation blueprint.

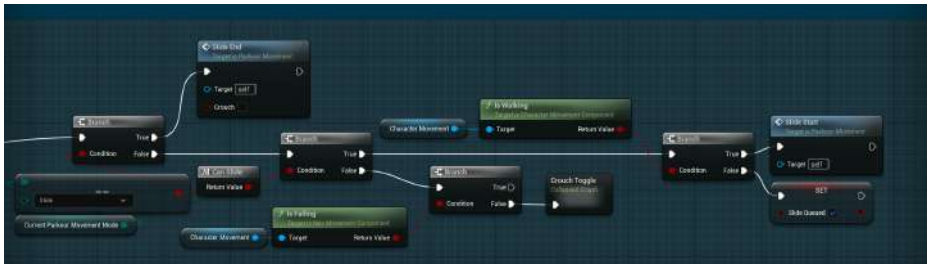

Fig. 16: Piece Crouch event in Blueprints (source: own elaboration)

Second the Crouch event (see figure 16) checks if the actor's current parkour state is none and forward input is higher then 0. If both of those statements are true and the current player state is walking, then the Slide is being performed, otherwise, the Slide is being Queued until the player hits the ground. The Slide start significantly decreases the actor ground friction, braking deceleration walking, increases max walking speed crouched and launches a character in the Slide direction vector that is calculated using a cross product of actor right vector ($\vec{r}$) and the ground normal ($\vec{n}$). The equation result is always a vector that is parallel to the ground.

$$\vec{s} = -\vec{r} \times \vec{n}$$

The ground normal ($\vec{n}$) is calculated using Hit Impact Normal of the following vector:

$$\vec{v} = -200 * \vec{u} - \vec{l}$$

Where $\vec{u}$ is actor up vector and $\vec{l}$ is actor location.

The Slide update on each tick calculates the floor influence on the player, to increase the character velocity in the direction the floor slope is angled. The calculate floor influence method returns 0 vector if vector up and floor normal are equal with a 0.01 tolerance. Otherwise, the floor influence is calculated using the equation below:

$$\vec{v} = \vec{u} \times \vec{n} \times \vec{n}$$

Where $\vec{u}$ is vector up and $\vec{n}$ floor normal.

The floor influence is then multiplied by floor influence force and applied to the character movement as an additional force. If the character velocity is lower than 300cm/s, the Slide End method is called, that resets the players characteristics and performs an uncrouch.

5.1.7 Coyote Time

It is a mechanic that allows the players to perform a regular jump, even a few frames after they last touched the ground. It was created to make the parkour more forgiving and aid the players during gameplay. For the regular jump, the **Jump()** function from the Movement Component was used. In order for the Coyote Time to work as intended, a custom code had to be added to the engine function that checks whether or not the jump can be performed. Its override can be seen in Listing 1.

```
1 bool ATheFallenSamuraiCharacter::CanJumpInternal_Implementation() const
2 {
3   return Super::CanJumpInternal_Implementation() || !bLockedJump;
4 }
```

Listing 1: Jump check required for the Coyote Time (source: own elaboration)

The boolean variable **bLockedJump** is set to false by default. However, some time after the player started to fall, it is set to true to lock the ability to perform the regular jump. When the player either lands or enters the wallrun state it is set back to false. The implementation of the following behaviour can be seen in Listing 2.

```
void ATheFallenSamuraiCharacter::OnMovementModeChanged(EMovementMode
    PrevMovementMode, uint8 PreviousCustomMode)
{
  Super::OnMovementModeChanged(PrevMovementMode, PreviousCustomMode);

  if (GetCharacterMovement()->MovementMode == MOVE_None)
  {
    ResetCoyoteProperties();
  }
  else if (GetCharacterMovement()->MovementMode == MOVE_Falling &&
    bFirstJump)
  {
      GetWorld()->GetTimerManager().SetTimer(CoyoteTimeTimer, this, &
    ATheFallenSamuraiCharacter::EnableJumpLock, CoyoteTime, false);
  }
}
```

Listing 2: Function managing the Coyote Time state (source: own elaboration)

5.2 Procedural Animation

The following section explores the technical implementation of procedural animation solutions employed in the game described. It highlights the motivation, encountered issues, and proposed solutions with a conclusion that presents what was achieved.

5.2.1 The motivation behind the system

When building a First Person Perspective game, authors had to ensure that the quality of the world presented matches a certain level of visual fidelity. Bringing the camera closer to the environment allows for creating much more visceral player experience by boosting the intimate feel of the in-game interactions. Yet, it also creates a bunch of significant challenges for the creators to tackle, as the lack of polish in some areas becomes much more noticeable than in, say, Third Person Perspective or Platformer games. In such case, one of the most significant aspects to consider, are the player's hand movement and animation which constantly are present in the viewport, no matter the level the player's are currently in. Since only a rather basic and crudely-crafted animations were at authors disposal - which if put in the game directly, without altering, would violate the rules

formed above - a system leveraging the Unreal Engine's procedural animation was created to solve both the visual and the utility issues that those animations exhibit. The two main goals set for the system were the following:

- Enhance the visual experience by procedurally altering hands transform to produce different poses based on the player actions.
- Improve the precision of the combat animations by refactoring their trajectory at runtime.

It is worth mentioning, that Rosen's talk *Animation bootcamp: An indie approach to procedural animation*[18] encouraged the authors to create a procedural animation system of their own. However, the case of the game described differed from the concepts presented in the talk, thus it should be considered mainly as an inspiration, not the source of solutions.

5.2.2 The tools

The solutions proposed below were implemented inside Unreal Engine's Control Rig.

5.2.3 Utility Improvements

Spine leaning - procedural aim-offset

As the animations at the authors disposal had a fixed position, a simple procedurally-calculated aim-offset[5] was employed to adjust animation rotation based on the player's Pitch[6]. For simplicity this solution was not implemented in Control Rig, but rather inside the AnimGraph of the Player Character Animation Blueprint. The main purpose of this system was to adjust the orientation of both hands so that the player can attack on multiple planes, not just the one where the default animations are playing. To achieve the following, a Transform Bone node was used to bind player's camera Pitch to the spine05 bone rotation. As shown in figure 17, this particular bone

[5] An asset that stores a blendable series of poses to help a character aim a weapon in the desired direction [source: `https://dev.epicgames.com/documentation/en-us/unreal-engine/creating-an-aim-offset?application_version=4.27`]

[6] The up-and-down rotation (relative to the Y-axis in Unreal Engine)

was chosen, since it is the highest bone in the skeleton hierarchy to which both arms are parented.
For the clarity, the system described in this section is presented on the default Unreal Engine mannequin that was later swapped with a game-ready, fully-detailed model.

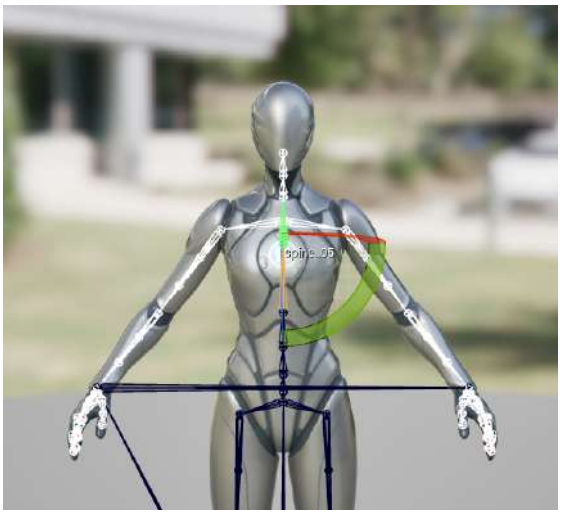

Fig. 17: Bones parented to spine05 - shown in white (source: own elaboration)

Rotation mode inside the Transform Bone node was set to "Add to Existing" as the default bone rotation should not be overridden, but rather added to. Also using the player's camera Y-axis rotation directly was impossible, since the World Coordinate System and Animation Coordinate System are different. Thus, the Pitch value had to be negated and used as a Roll(X-axis rotation) while setting the bone rotation. As a result, if the player pans the mouse, both arms can adjust, so that they always remain in the viewport.

Hand trajectory adjustment

One of the critical systems employed in the procedural animation system is the attack animation adjustment. This system, along with spine leaning, can benefit the player significantly.

As stated previously, the animations were not ideal, particularly in the case of attack animations. These often appeared to play only on the right side of the screen, which was unacceptable. In a precision-driven game, this issue could quickly frustrate players, as they would feel they were losing due to imprecise animations rather than their own mistakes. With that in mind a following system was created, allowing any attack animations to cut through an arbitrary point, by altering the trajectory at runtime. To achieve that, a Control[7] parented to the head bone was introduced. It serves as a target point during the calculations and was crucial while implementing the auto-aim system described in the 5.3.5 section. As depicted in figure 18, the main objective was to calculate a vector(shown in yellow) that would shift player's attack from the default incorrect plane(marked red) onto the one that contains the target point(marked green).

Fig. 18: The difference between attack planes (source: `https://openart.ai/community/CD8FkQnmRXNFVO9vQ9jJ`)

[7] Point in the animation space with its own transform. It can be parented to any bone in the skeleton to form a dynamic hierarchy. It can also be used as a utility feature to represent a simple reference transform

The vector in question was determined simply by calculating a vector rejection, described in [13], with the following equation:

$$\vec{o} = \vec{t} - (\vec{t} \cdot \hat{\vec{v}})\hat{\vec{v}}$$

where $\hat{\vec{v}}$ is the normalized right hand velocity vector and $\vec{t}$ is the vector from the hand to the target point. By subtracting the projected portion of vector $\vec{t}$ from the vector $\vec{t}$ itself, an offset vector $\vec{o}$ can be determined. It is perpendicular to the incorrect trajectory and has a length equal to the distance between both trajectories, thus it can be used to offset the right hand position.
However, the calculated offset adjusts the animations only vertically. To ensure the symmetry relative to the Target Point, a precalculated vector between the starting and ending location of the right hand must be used. That calculation is conducted once, during the Combat System initialization process described in the section 5.3.2.
The horizontal offset ($\vec{h}$) is calculated from the following equation:

$$\vec{h} = \vec{t} - (\vec{r} + 0.5\vec{a})$$

where $\vec{a}$ is the attack vector, $\vec{r}$ is the right hand's current location (in the animation space), and $\vec{t}$, as before, is the Target Point location. Now, the X component of the $\vec{h}$ can be added to the offset vector $\vec{o}$ and used to correctly shift the right hand. It is important to store and compute all of the above vectors in the animation space, not only for convenience, but for a small optimization. This way, the attack vector, for instance, does not have to be computed every time the character changes its orientation.

By repeating the following process for every frame of the current attack animation and applying the new position with the Two-Bone IK[8] node allows for an attack to be performed in any direction, making the combat more flexible and slightly more forgiving.

5.2.4 Visual Improvements

[8] The Two Bone IK control applies an Inverse Kinematic (IK) solver to a 3-joint chain, such as the limbs of a character. d[source: `https://dev.epicgames.com/documentation/en-us/unreal-engine/two-bone-ik?application_version=4.27`]

Right hand adjustment

As the authors drew much inspiration from the Ghostrunner game, along with the samurai culture in general, it was decided that the sword will be held with both hands. Figure 19 shows the reference that was used while creating this system. However, the chosen animations were originally created for the medieval setting where the left hand was responsible for wielding the shield and the right hand for attacking with the sword. Such approach was unacceptable since not only did it collide with the author's vision but also the animations showed a bunch of different issues.

Fig. 19: Ghostrunner II sword grab (source: own elaboration)

First of which was the sword positioning. Compared to the Ghostrunner most of the right hand should be visible inside the viewport, whereas in the default idle animation, hand was animated extremely low exposing mostly the upper part of the Katana blade instead of the handle. This issue can be observed in figure 20.

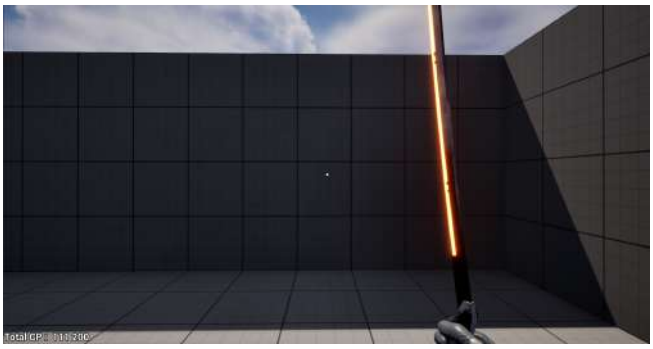

Fig. 20: The incorrect position of the Right Hand (source: own elaboration)

In order to solve this problem, a Control(see figure 21) was used as an easy, user-friendly gizmo allowing the authors to position the right hand in the correct spot and adjust if necessary. To mimic the Ghostrunner even further, introduced Control also holds the information about the hand rotation offset, allowing to lean the hand slightly to the right.

Fig. 21: Right hand Control in Control Rig editor (source: own elaboration)

Next, in order to correctly position the right hand along with the whole right arm, the Two-Bone IK node was used that allows to realistically set the transform of a given bone chain, while respecting the animation constraints. A number of features like the Pole Vector[9] or the stretching factor were used to produce more grounded results - the former was treated as the direction pointing to the upper-right(making the elbow less hanging) and the latter was entirely disabled.
The Two-Bone IK node, however, is not additive, thus naively setting the hand transform to the Control transform ignored the animation data and produced noticeable artifacts when playing any of the locomotion animations.

The first solution was to calculate the difference between the right hand's position in the previous frame and the current one, offset the Control Point location by that amount, and only then, set the transform. Unfortunately, this introduced horrible choppiness to the animations caused by the lack of interpolation between the updates, thus this solution was abandoned.

Much better approach involved adding the second Control(shown in figure 22, marked as red), setting its position to the initial right hand position in the idle animation and calculating the offset once, when the Construction Event[10] is called. Now, by using the Two-Bone IK node to move the right hand by that offset, the animation can play smoothly, but around the user-defined point without losing any of the animation data.

[9] Either a direction or a position vector needed for the Inverse Kinematics calculations, used to determine the point (or a general direction) towards which affected bones should point
[10] An Event that is fired every time given blueprint/object is initialized

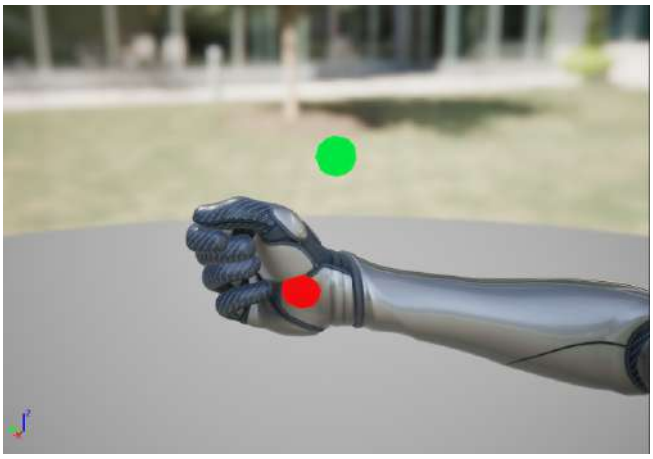

Fig. 22: Reference(red) and screen(green) Control positions (source: own elaboration)

Apart from the location, the desired right hand rotation is also taken into account to slightly lean the hand to the right. As the Control holds not only the location but also the rotation data, initially chosen approach was to simply use its quaternion rotation directly in the IK calculations. However, this presented an issue caused by the previously described procedural aim-offset system which binds the spine05 rotation to the camera's pitch. As both, the right and the left hand are parented to the bone mentioned above, rotating the camera up or down influenced the right hand rotation around its wrist which produced terrible results.
Since procedural aim-offset is crucial to the fast-paced and precise gameplay and the hand rotation is just a visual feat, another solution had to be employed.

Significantly better approach involved breaking both the Control's and right hand's quaternion into the Euler Angles in the ZYX rotation order to later combine it into the resulting quaternion where the rotation along Y axis is set based on the Control's Y rotation and rotation along X and Z axis are set based on the hand's X and Z rotation. The entire approach described above allowed the authors to customize the right hand rotation and position to suit the game's

visual needs without worrying about breaking other systems that support the critical parts of gameplay.

Left hand adjustment

The next step is to adjust the left hand so that the player character can believably grab the sword with both hands. Thanks to the original purpose of the left hand's animations - to hold the shield - the fingers in every animation were already properly clenched so creating additional animations or procedural solutions for the individual fingers was not required. Thus the problem became only a matter of offsetting the hand so that it is located below the right hand and closed around the hilt of the sword.

The first step in implementing the following system is to ensure that the left hand calculations are conducted after the right hand is properly positioned. The correct order of those operations is extremely important as setting the left hand first and then the right would cause noticeable visual incoherency as the left hand would base its calculations on the right hand's previous transform.

The process of calculating the resulting left hand transform produces couple more challenges to face:

- The hand bone is not located in the palm but rather in the center of its wrist, thus using its location becomes more tricky;
- The rotation of either the right hand or the sword has to be acknowledged while positioning and rotating the left hand to prevent visual artifacts

In order to solve the first problem, a Skeleton Socket[11] was introduced. Thanks to its dedicated editor(shown in figure 23) that allows for free positioning, setting the custom hand position became significantly easier.

[11] Socket is a dedicated attach point within the hierarchy of the Skeleton, which can be transformed relative to the Bone it is parented to. Once set up, various objects can be attached to the Socket such as weapons or other actors. [source: `https://dev.epicgames.com/documentation/en-us/unreal-engine/skeletal-mesh-sockets-in-unreal-engine`]

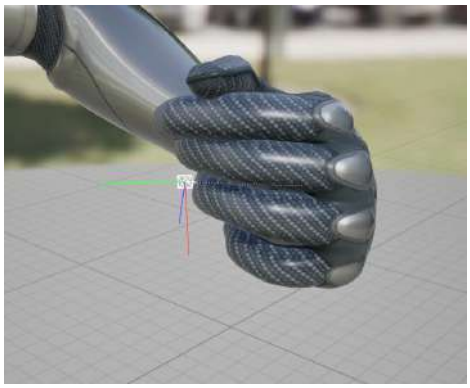

Fig. 23: Left hand's Socket in the Skeletal Mesh editor (source: own elaboration)

The sockets were added to both hands - left one specifically to aid the procedural animation system, the right one however was also used as an attachment point for the Katana Actor which is described later in the Combat System Initialization section 5.3.2.

In order to calculate the position of the palm of either hand in the animation space, the bone space transform of the socket is multiplied by the hand transform in the animation space. Now, the quaternion from the newly calculated right hand transform holds the information about the axis and the angle of the right hand rotation. By rotating an arbitrary vector (Katana's Forward Vector in this case, since it is parallel to the blade) by that quaternion and adding it to the right hand position, a proper location for adjusting the left hand can be determined. The last step is to offset this location by the difference between the palm and the wrist . If this step was omitted, the sword would seem to be "growing out" of the hand, shown on the figure 24. Once the position is calculated, it can be fed as an input into the Two-Bone IK node that is responsible for positioning the left hand.

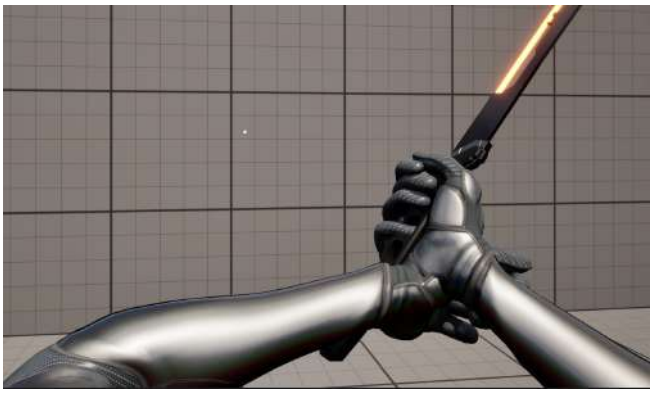

Fig. 24: Applying the left hand transform without the offset (source: own elaboration)

Unfortunately, the second problem appears - a noticeable clipping, that can be spotted in the figure 25. The culprit here lies in directly setting the left hand quaternion without accounting for the right hand's rotation.

Fig. 25: Thumb clips into the palm of the right hand (source: own elaboration)

The solution involves taking the right hand Control quaternion and calculating the difference between the left hand Control quaternion. Now, by swapping the rotation axis to reflect the left hand coordinate system, the resulting quaternion can be used as a correction for the left hand rotation and input into the Two-Bone IK node. Figure 26 compares the result of this solution with the default hand rotation.

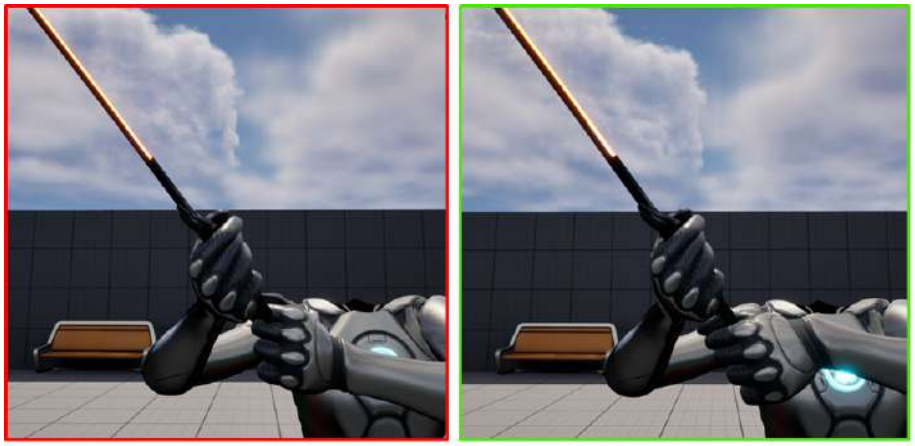

Fig. 26: Default rotation on the left and corrected on the right (source: own elaboration)

5.2.5 Final Result

The entire Procedural Animation System is a notable feature, that the authors are proud of. While certainly it is not perfect and could be improved to employ more robust calculations, it still serves its purpose well and allowed to work around the lack of the animations the authors had in mind. The result of the system - each of the solutions described above combined - is shown in the figure 27.

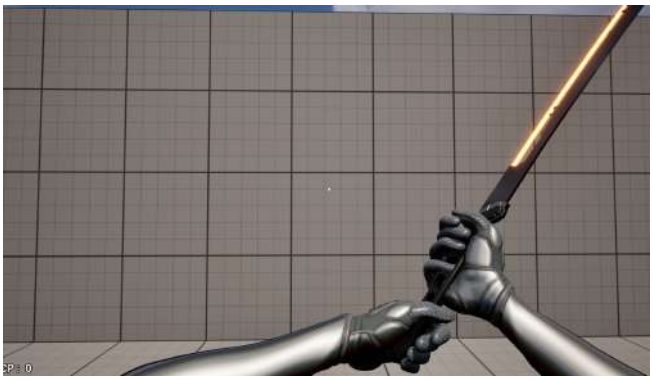

Fig. 27: Left and right hand solvers combined, producing the final result (source: own elaboration)

5.3 Combat system

Combat system is one of the two main mechanics of the game described. It is responsible for every single action that the player can take to eliminate his adversaries. The following section delves deeply into its C++ implementation.

5.3.1 Architecture

Entire Combat System was implemented inside the UCombatSystemComponent class that derives from UActorComponent, an Unreal Engine class that cannot exist in the game world by itself and can only be attached to other AActor class. In this case, UCombatSystemComponent is attached to the player character class.

5.3.2 Initialization

The **InitializeCombatSystem()** function is called within the player character class, once the Begin Play event is fired. It is responsible for a number of actions that include:

- Binding the curves and update functions to the FTimeline
- Setting the initial attack animation
- Setting up both arrays of Animation Montages: for attacks and counter attacks

- Spawning the AKatana actor and attaching it into the right hand's Skeleton Socket
- Adding listeners to the animation event(shown in Listing 3)

```
AnimInstance = playerCharacter->GetMesh()->GetAnimInstance();

AnimInstance->OnPlayMontageNotifyBegin.AddDynamic(this, &
    UCombatSystemComponent::PlayMontageNotifyBegin);
AnimInstance->OnPlayMontageNotifyEnd.AddDynamic(this, &
    UCombatSystemComponent::PlayMontageNotifyEnd);
AnimInstance->OnMontageBlendingOut.AddDynamic(this, &
    UCombatSystemComponent::PlayMontageFinished);
```

Listing 3: Dynamic bindings to the animation notify events (source: own elaboration)

- Calculating needed variables for the Teleport System.
- Calculating the Attack Vector for every attack animation used by procedural animation and the directional camera shake.

The most important step during the system initialization is the AKatana class setup depicted in Listing 4. Once the actor itself is spawned with the correct spawn parameters, the ending blade socket is offset. This step is needed as the collision tracing is performed between the two blade sockets. In order to give players the advantage in combat, the end socket is moved farther away from the blade tip, making the collider on the Katana twice as big as the blade itself. Once this step is completed, the AKatana actor is attached to the KatanaSocket positioned in the player's right hand's palm.

```
FActorSpawnParameters KatanaSpawnParams;
KatanaSpawnParams.SpawnCollisionHandlingOverride =
    ESpawnActorCollisionHandlingMethod::AlwaysSpawn;
KatanaSpawnParams.TransformScaleMethod = ESpawnActorScaleMethod::
    MultiplyWithRoot;

Katana = GetWorld()->SpawnActor<AKatana>(KatanaActor, player->GetTransform
    (), KatanaSpawnParams);
Katana->OffsetTraceEndSocket(KatanaBladeTriggerScale);

HitTracer = Katana->HitTracer;

EAttachmentRule KatanaAttachRules = EAttachmentRule::SnapToTarget;

Katana->K2_AttachToComponent(player->GetMesh(), "KatanaSocket",
  KatanaAttachRules, KatanaAttachRules, KatanaAttachRules,
```

```
14    true);
```

Listing 4: AKatana actor spawn and attachment (source: own elaboration)

5.3.3 Attack Implementation

The only external solution in the attack code is the DidItHit plug-in, that allows for a very basic collision tracing on the sword blade. The plug-in itself was described in detail in the Programming techniques section 4.4.

An **Attack()** function is bound to the Left Mouse Button. Once called, the state of the Combat system is checked, as the attack can also be used when The Super Ability described in section 5.3.7 is active. If the attack can be performed(i.e. any other attack animation is not playing), another function shown in Listing 5 is called.
Each of the attack animations has an Animation Notify Window. Once the window begins, the collision trace starts by calling the **ToggleTraceCheck()** function from the DitItHit plug-in.
To make the game more responsive players are allowed to cancel the current attack by pressing the Left Mouse Button again, but only after the Notify Window(along with the collision trace) has ended, without the need for waiting until the current animation has finished playing.

```
1  void UCombatSystemComponent::SwingKatana()
2  {
3    HitTracer->ClearHitArray();
4
5    bIsAttacking = true;
6    bInterputedByItself = false;
7    bShouldIgnoreTeleport = false;
8
9    float AttackMontageStartPercent = .21f;
10   AnimInstance->Montage_Play(NextAttackData.AttackMontage,
       AttackSpeedMultiplier, EMontagePlayReturnType::MontageLength,
       AttackMontageStartPercent);
11
12   CurrentAttackData = NextAttackData;
13   NextAttackData = DetermineNextAttackData();
14
15   GetWorld()->GetTimerManager().SetTimer(EnemiesTraceTimerHandle, this,
16     &UCombatSystemComponent::GetEnemiesInViewportOnAttack,
```

```
    1 / 60.f, true);
}
```

Listing 5: Function called once the attack is initiated (source: own elaboration)

For each frame during the attack function **ProcessHitResult()** is called which checks what was hit during the Katana swing. Listing 6 shows its implementation.
The Katana slice plane normal is calculated which is needed later for shield destruction and procedural Static Mesh slicing provided any sliceable Actor was also hit. Other attack properties are encapsulated inside a data structure and sent down to the enemy. If the enemy hit is alive, the hit impact camera shake is played.
It is also important to test whether the enemy is not protected by a shield. If so, no damage is applied and a hit reaction response is handled.

```
void UCombatSystemComponent::ProcessHitResult(const FHitResult& HitResult)
{
  //slice plane code
  FVector HandVelocity = playerCharacter->GetMesh()->GetBoneLinearVelocity
    ("hand_r");
  FVector PlaneNormal = Katana->GetBladeWorldVector().Cross(HandVelocity);
  PlaneNormal.Normalize();

  FVector TrueImpactPoint = Katana->KatanaMesh->GetSocketLocation("Middle"
    );

  AActor* HitActor = HitResult.GetActor();
  if (auto Enemy = Cast<ABaseEnemy>(HitActor))
  {
    if (!CurrentAttackData.bIsTeleportAttack && Enemy->bOwnsShield)
    {
      auto ToPlayerNormalized = (playerCharacter->GetActorLocation() -
    HitActor->GetActorLocation()).GetSafeNormal();
      if (CheckIsShieldProtected(ToPlayerNormalized, HitActor->
    GetActorForwardVector()))
      {
        ProcessHitResponse(ShieldHitImpulse, HitResult.ImpactPoint);
        return;
      }
    }

    UGameplayStatics::SpawnEmitterAtLocation(GetWorld(),
      BloodParticles, Enemy->GetMesh()->GetBoneLocation("head"), FRotator
    (0), BloodScale);
```

```
25
26      FCombatHitData KatanaHitResult;
27      KatanaHitResult.ActorCauser = playerCharacter;
28      KatanaHitResult.ImpactLocation = HitResult.ImpactPoint;
29      KatanaHitResult.CutPlaneNormal = PlaneNormal;
30      KatanaHitResult.CutVelocity = HandVelocity;
31      KatanaHitResult.bSuperAbilityKill = SuperAbilityTargetsLeft >= 0;
32
33      if (!Enemy->HandleHitReaction(KatanaHitResult))
34      {
35        PlayerCameraManager->StopAllCameraShakes();
36
37        PlayerCameraManager->PlayWorldCameraShake(GetWorld(),
38          HitCameraShake,
39          playerCharacter->GetActorLocation(),
40          1, 500, 1);
41      }
42    }
43    else if (auto SlicableActor = Cast<ASlicableActor>(HitActor))
44    {
45      SlicableActor->SliceActor(PlaneNormal, HitResult.ImpactPoint);
46    }
47  }
```

Listing 6: Hit reaction handling implementation (source: own elaboration)

5.3.4 Directional Camera Shake

The Directional Camera Shake system allowed to create a greater impact and satisfaction even from the basic sword swinging. It allows to rotate the camera in the direction of the attack animation thanks to the pre-calculated attack vector between the right hand's position in the beginning and in the end of the sword swing. Such functionality was achieved by creating a custom shake class called **UDirectionalCameraShake** that derives from the Unreal Engine's **UCameraShakeBase** class. It was needed to pass the current attack vector into the Shake Pattern, once the attack was initiated.
The core functionality is implemented inside the custom shake pattern called **UDirectionalMixedShakePattern**, a subclass of the **USimpleCameraShakePattern**. The implementation of its update function can be seen in Listing 7. It allows to blend the value from the user-defined curve to rotate the camera in the attack di-

rection along with applying the slight location offset in direction perpendicular to the attack vector in a sinusoidal pattern.

```
void UDirectionalMixedShakePattern::UpdateShake(float DeltaTime,
    FCameraShakeUpdateResult& OutResult)
{
  ShakeCurrentTime += DeltaTime;

  float CurveCurrentTime = ShakeCurrentTime / Duration;
  OutResult.Rotation.Yaw = InterpCurve->GetFloatValue(CurveCurrentTime) *
    RotationAmplitude * ShakeLocalDirection.Y;
  OutResult.Rotation.Pitch = InterpCurve->GetFloatValue(CurveCurrentTime)
    * RotationAmplitude * ShakeLocalDirection.Z;

  OutResult.Location = WaveVectorShake.Update(DeltaTime, 1.f, 1.f,
    WaveVectorTime) * ShakePerpDirection;
  OutResult.Location.X = 0.f; // never shake forward

}
```

Listing 7: Camera Shake Pattern update (source: own elaboration)

A couple of other approaches were tested - sampling Perlin noise either along that vector or to create an arbitrary shake position - however, the approach presented above proved to be the smoothest and most visually pleasing.

5.3.5 Auto-aim Implementation

As mentioned a couple of times in the Procedural Animation section, the combat animations were very far from ideal, which was unacceptable in a game where both the precision and the impact play a major role during combat. With that in mind, a simple yet effective auto-aim system was created.

Once the attack is initiated, the timer is started that can be seen at the bottom of the Listing 5. It repeatedly calls the **GetEnemiesInViewportOnAttack()** function until either the attack animation has finished or an enemy was hit. It traces for all the enemies in front of the player by using a **BoxTraceMultiForObjects()** function from **UKismetSystemLibrary**. By performing a linear search on all of the enemies hit by the box trace, the one closest to the player is chosen. If the distance to the target enemy is less or equal to the katana collider length, function **GetAutoAimOffset()** is called to calculate how to offset the target point relative to

the centre of the screen. Its implementation can be see in Listing 8 below.

```
FVector UCombatSystemComponent::GetAutoAimOffset(const FVector&
    PlayerLocation, const FVector& EnemyLocation, const FVector&
    PlayerForwardVector, const FVector& PlayerUpVector)
{
  auto ToEnemy = EnemyLocation - PlayerLocation;

  FVector PlayerRightVector = PlayerUpVector.Cross(PlayerForwardVector);

  float MaxAbsoluteYOffset = 45.f;
  float TargetPointYOffset = FMath::Clamp(ToEnemy.Dot(PlayerRightVector),
    -MaxAbsoluteYOffset, MaxAbsoluteYOffset);

  float TargetPointZOffset = FMath::Clamp(ToEnemy.Dot(PlayerUpVector),
    -17, 7);

  return FVector(0.f, TargetPointYOffset, TargetPointZOffset);
}
```

Listing 8: Implementation of the offset calculation (source: own elaboration)

The result of that function is later passed to the Control Rig for the Trajectory Adjustment system. The correct offset is calculated by projecting the vector to enemy on the the player's right and up vectors and clamping it, to prevent it from exceeding the borders of the screen. This way, all of the attack animation, with the help of the system inside the Control Rig, can be shifted in any direction, making the combat more flexible.

5.3.6 Combat Teleport Implementation

Surprisingly, even with the larger Katana collider and the Auto-aim System, players had issues with killing the enemies. In the First Person Perspective, often the enemy seems to be close enough, yet in reality, even the doubled collider cannot overlap with it. To solve this issue, a Combat Teleport system was introduced.
It is a guided dash, that is performed automatically if the closest enemy in the viewport cannot be reached with the standard attack while simultaneously is not farther than a specified distance threshold. In other words - when the auto-aim system would be ineffective, the teleport system is used. The decision whether to use the teleport

or not, is also made in the **GetEnemiesInViewportOnAttack()** function described above.

Considering that the teleport triggers automatically, the system had to be written thoughtfully, so that it aids the player, instead of causing frustration. To ensure its correct behaviour, there are several steps to take before the teleport begins.

Once the target enemy has been chosen, teleport position must be determined. To ensure that moving enemies are also valid targets, not only is the enemy's current position taken into account, but also its velocity. Once the future position of the target is determined, a teleport position is selected as presented in the figure 28. To avoid teleporting directly into the enemy, an offset equal to the 70 percent of the length of the Katana collider is applied.

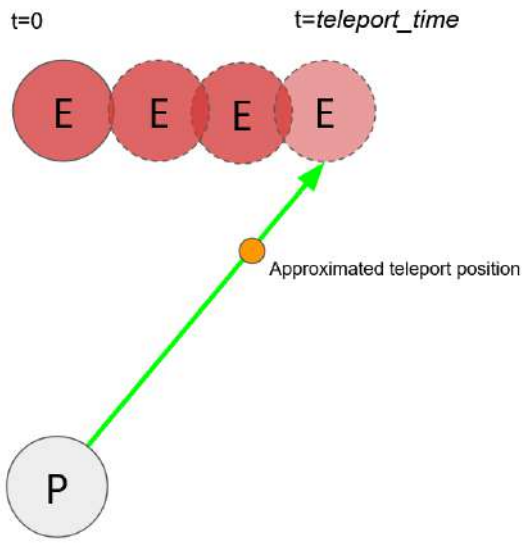

Fig. 28: Teleport destination point (source: own elaboration)

Once the location is chosen, a couple of tests has to be performed.

Visibility check tests whether the enemy is visible by casting two line traces from the players head: to enemy head and to the

enemy's center of mass. The implementation can be seen below, in Listing 9.

```
bool UCombatSystemComponent::CheckIsTeleportTargetObscured(ABaseEnemy*
    Enemy)
{
  FVector EyeStart = playerCharacter->GetMesh()->GetBoneLocation("head");
  FVector EyeEnd = Enemy->GetActorLocation();
  FHitResult EyeOutHit;

  FCollisionQueryParams CollisionParams;
  CollisionParams.bTraceComplex = true;
  CollisionParams.AddIgnoredActor(playerCharacter);
  CollisionParams.AddIgnoredActor(Katana);
  CollisionParams.AddIgnoredActor(Enemy);

  bool bEyeToCenterHit = GetWorld()->LineTraceSingleByChannel(EyeOutHit,
    EyeStart, EyeEnd, ECollisionChannel::ECC_Visibility, CollisionParams);

  EyeEnd.Z += Enemy->GetCapsuleComponent()->GetScaledCapsuleHalfHeight();

  bool bEyeToEyeHit = GetWorld()->LineTraceSingleByChannel(EyeOutHit,
    EyeStart, EyeEnd, ECollisionChannel::ECC_Visibility, CollisionParams);

  return bEyeToCenterHit && bEyeToEyeHit;
}
```

Listing 9: Visibility check implementation (source: own elaboration)

By performing the **Geometry check**, system can verify that the player will not be stuck inside some geometry once the teleport is finished. This is an important thing to consider, as the collision on the player is disabled while in the teleport state. This allows the system to be more flexible, as the player will never get stopped by anything while teleporting.
The test itself consist of a simple **CapsuleTraceSingleForObjects()** trace with the player's capsule dimensions.

The **Ground check** ensures that the player will not fall through the ground. It is relatively simple, as only one downward trace is cast. If the static, immovable geometry is hit, the check passes and the code continues.

Now, if the target is not obscured and the teleport position is safe then there is one more thing to consider. Even if the ground was detected, it does not mean that the sword could cut the enemy. There is a special case, when the player is teleported to a location

either above or below the target. On the other hand, such a case should not result in a failure, as on the levels there may be a number of spaces (like stairs, or any other uneven surface), where the teleport should be activated. Thus, the final test is performed - **Cut Check** - that determines whether the enemy will be cut. If not, for a greater flexibility, it tries to use the Auto-aim System to procedurally adjust the current attack animation. Implementation of this functionality can be seen in Listing 10.

```
auto FeetToHead = playerCharacter->GetMesh()->GetBoneLocation("head") -
    playerCharacter->GetMesh()->GetBoneLocation("root");
auto CombatPoint = GroundHit.Location + FeetToHead;

TargetPointOffset = GetAutoAimOffset(PlayerDestinationForTeleport,
    EnemyLocationOverTime,
  RotationToEnemy.Vector(), playerCharacter->GetActorUpVector());
CombatPoint += RotationToEnemy.Vector() * CharacterArmsLength +
    TargetPointOffset;

FVector KatanaStart = CombatPoint;
FVector KatanaEnd = KatanaStart + RotationToEnemy.Vector() *
    KatanaTriggerLen;

float EnemyBottom = EnemyLocationOverTime.Z - Enemy->GetCapsuleComponent()
    ->GetScaledCapsuleHalfHeight();
float EnemyTop = EnemyLocationOverTime.Z + Enemy->GetCapsuleComponent()->
    GetScaledCapsuleHalfHeight();

if (!UKismetMathLibrary::InRange_FloatFloat(KatanaStart.Z, EnemyBottom,
    EnemyTop) && !UKismetMathLibrary::InRange_FloatFloat(KatanaEnd.Z,
    EnemyBottom, EnemyTop))
  return false;
```

Listing 10: Determining whether Katana will cut the enemy (source: own elaboration)

Another crucial element is the synchronization of the animation play-rate with the teleport time. If the attack animation would finish sooner than the teleport, there is a high chance that the enemy would be completely missed, leaving the player at a disadvantage. To prevent that from happening, another AnimNotify was introduced, that can be seen in figure 29. It allows the system to either slow down or speed up the current animation so that, once the teleport finishes, the animation is exactly in the spot marked by the AnimNotify.

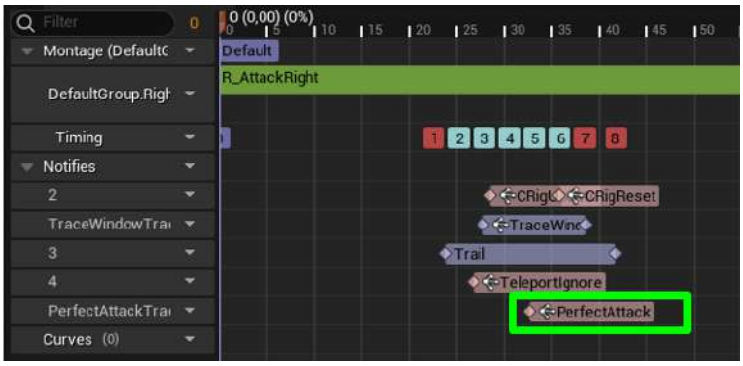

Fig. 29: The "perfect" attack time AnimNotify - marked in green (source: own elaboration)

As the notify execution time does not change, it can be read and cached when the Combat System initializes. When the teleport is activated, it calculates the time to that notify:

$$TimeLeft = NotifyTime - CurrentAnimTime$$

and substitutes the result into the following equation:

$$PlayRate = \frac{TimeLeft}{TeleportTime}$$

The resulting ratio is the new animation play rate. To prevent the negative subtraction results, once the current animation passes the "Teleport Ignore" AnimNotify(that can also be seen in figure 29) teleport system is not initiated. It also allows to avoid a case, when the teleport is started too late in the animation, and the player is teleported after the collision tracing on Katana blade was turned off.

Now, the teleport can begin, by interpolating the location, rotation and FOV along specified curves via the FTimeline. Location is the point calculated and tested above and the rotation is set by the code in the Listing 11. The Field Of View is interpolated between the camera default FOV and a user-defined variable.

```
1 PlayerOnTeleportRotation = playerCharacter->GetControlRotation();
2 RotationToEnemy = playerCharacter->GetControlRotation();
3
4 FVector LookAtEnemyLocation = EnemyLocationOverTime;
```

```
LookAtEnemyLocation.Z -= Enemy->GetCapsuleComponent()->
    GetScaledCapsuleHalfHeight() * .3f; // so that player looks a bit down

FRotator FaceEnemyRotation = (LookAtEnemyLocation -
    PlayerDestinationForTeleport).Rotation();
FRotator Delta = UKismetMathLibrary::NormalizedDeltaRotator(
    FaceEnemyRotation, PlayerOnTeleportRotation);

if (!ValidationRules.bUsePitch)
  Delta.Pitch = 0;

RotationToEnemy += Delta;
```

Listing 11: Calculating player rotation after the teleport (source: own elaboration)

The function that handles the teleport activation, once all above is completed, can be reviewed in the Listing 12.

```
void UCombatSystemComponent::TeleportToEnemy(float TeleportDistance)
{
  bInTeleport = true;

  CurrentAttackData.bIsTeleportAttack = true;

  PlayerCameraFOV = PlayerCameraManager->GetFOVAngle();

  float NormalizedTeleportTime = UKismetMathLibrary::MapRangeClamped(
    TeleportDistance, KatanaTriggerLen,
     TeleportTriggerLength, MinTotalTeleportTime, MaxTotalTeleportTime);

  TeleportTimeline.SetPlayRate(1.f / NormalizedTeleportTime);

  auto CurrentAttackMontage = CurrentAttackData.AttackMontage;
  float TimeToPerfectAttack = CurrentAttackData.PerfectAttackTime -
    AnimInstance->Montage_GetPosition(CurrentAttackMontage);
  float AcctualPlayRate = TimeToPerfectAttack / NormalizedTeleportTime;

  AnimInstance->Montage_SetPlayRate(CurrentAttackMontage, AcctualPlayRate)
    ;

  playerCharacter->GetCharacterMovement()->DisableMovement();
  playerCharacter->GetController()->SetIgnoreLookInput(true);
  playerCharacter->GetCapsuleComponent()->SetCollisionEnabled(
    ECollisionEnabled::NoCollision);

  OnIFramesChanged.Broadcast(true);

  TeleportTimeline.PlayFromStart();
```

```
}
```

Listing 12: Function that handles teleport activation (source: own elaboration)

5.3.7 Super Ability Implementation

The Super Ability can be thought of as the more advanced iteration of the Teleport system described above.
Once a certain amount of Combo Points have been collected, the player can enter the targeting mode(presented in the figure 30) that highlights nearby enemies. Any enemy can be chosen as the target by simply looking in its direction and pressing Left Mouse Button to initiate a long-distance teleport. Up to four enemies can be killed this way.

Fig. 30: Super Ability Targeting Mode(source: own elaboration)

Once the E key is clicked the function seen in Listing 13 is called.

```
void UCombatSystemComponent::SuperAbility()
{
  if (SA_State == SuperAbilityState::WAITING)
  {
    CancelSuperAbility();
    return;
  }
```

```
  else if (SA_State != SuperAbilityState::NONE)
    return;

  if (ComboSystem->AbilityComboPoints < ComboSystem->SuperAbilityCost)
  {
    OnSuperAbilityCalled.Broadcast(false, "Not enough Combo Points");
    return;
  }

  SuperAbilityTargetsLeft = SuperAbilityTargetLimit;

  GetWorld()->GetTimerManager().SetTimer(SuperAbilityTimerHandle, this, &
    UCombatSystemComponent::ExecuteSuperAbility, 1 / 60.f, true);
}
```

Listing 13: Function that handles teleport activation (source: own elaboration)

In addition to the system setup, it starts a timer that repeatedly calls the **ExecuteSuperAbility()** function. Its main purpose is to scan the player's surrounding with a simple **SphereTraceMultiForObjects()** for nearby visible enemies. Next, the enemy that the player is looking at is stored and used as the Super Ability target. The way it is implemented can be seen in Listing 14. If there is at least one valid enemy target, the system enters a **Waiting** state. Otherwise, the Super Ability is automatically cancelled.

```
TArray<FHitResult> HitResults;
UKismetSystemLibrary::SphereTraceMultiForObjects(GetWorld(), Start, Start,
    MaxJumpRadius, ObjToTrace,
  true, Ignore, EDrawDebugTrace::None, HitResults, true);

float MaxDot = -1;
ABaseEnemy* Target = nullptr;

int ObscuredCounter = HitResults.Num();

for (auto HitResult : HitResults)
{
  auto Enemy = Cast<ABaseEnemy>(HitResult.GetActor());
  if (!Enemy)
    continue;

  if (CheckIsTeleportTargetObscured(Enemy))
  {
    ObscuredCounter--;
    continue;
  }
```

```
  else
  {
    Enemy->GetMesh()->SetCustomDepthStencilValue(3);
    PostProcessSA_Targets.Add(Enemy);
  }

  FVector ToEnemy = Enemy->GetActorLocation() - playerCharacter->
    GetActorLocation();
  float dot = playerCharacter->GetControlRotation().Vector().Dot(ToEnemy.
    GetSafeNormal());

  if (dot >= .97f)
  {
    if (dot > MaxDot)
    {
      MaxDot = dot;
      Target = Enemy;
    }
  }
}
```

Listing 14: Process of finding the right Super Ability target (source: own elaboration)

As in the case of the base teleport, enemy has to be validated. Here, however, the check is more advanced as all the possible spots around the target are checked, not the one in front of it. To achieve that, an options data, seen in Listing 15 is passed to the validation function allowing for the greater customization of the testing rules.

```
USTRUCT(BlueprintType)
struct FValidationRules
{
  GENERATED_BODY()

  bool bUseDebugPrint = false;
  EDrawDebugTrace::Type DrawDebugTrace = EDrawDebugTrace::None;

  float GroundTraceDepth;
  bool bUsePitch = true;
  bool bUseLazyCheck = true;
  int ChecksSampleScale = 1; //how granular the checks are placed: bigger
    number -> they are more "packed"
  bool bShouldIgnoreShields = false;
};
```

Listing 15: Validation Rules struct (source: own elaboration)

Listing 16 presents how the validation rules can be used when checking the current teleport target.

```
if (Target)
{
  FValidationRules ValidationRules;
  ValidationRules.bUseLazyCheck = false;
  ValidationRules.ChecksSampleScale = 2;
  ValidationRules.bShouldIgnoreShields = true;

  bool bIsValidTarget = ValidateTeleportTarget(Target, ValidationRules);
  if (SuperAbilityTarget != Target)
  {
    if (SuperAbilityTarget)
      SuperAbilityTarget->SetEnableTargetWidget(false);

    if (bIsValidTarget)
    {
      Target->SetEnableTargetWidget(true);
      SuperAbilityTarget = Target;
      OnSuperAbilityTargetAcquired.Broadcast(true);
    }
  }
}
```

Listing 16: Example of validation (source: own elaboration)

If the Left Mouse Button is clicked and the target is valid, the system state changes to **Teleporting**, and initiates the base teleport described above. The process is repeated until one of the following conditions is met:

- There are no enemies left
- Player cancels the ability manually by pressing the E key again
- Target limit has been reached

The Super Ability system strongly cooperates with the Combat UI. When initiated, the default crosshair is replaced and icons representing the remaining targets appear. The mock-up for the Combat UI is shown in the figure 31. For each Super Ability kill, the UI refreshes by animating the icons mentioned.

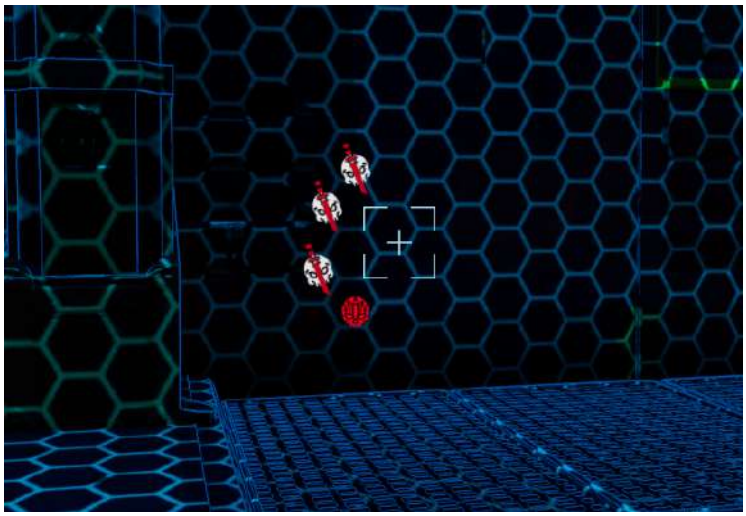

Fig. 31: Targeting UI showing one kill left(source: own elaboration)

To ensure flawless and flexible communication between UI and Combat System, an Observer Pattern was used, described in 4.4 section. In Unreal Engine it is implemented with Events and Delegates - their declaration is shown in Listing 17.

```
1 DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(FOnEnemiesLeftChanged, int,
    EnemiesLeft);
2 DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(FOnSuperAbilityTargetAcquired,
     bool, bFoundNewTarget);
3 DECLARE_DYNAMIC_MULTICAST_DELEGATE_TwoParams(FOnSuperAbilityCalled, bool,
    bWasSuccess, FString, FailReason);
4 DECLARE_DYNAMIC_MULTICAST_DELEGATE(FOnSuperAbilityCancelled);
```

Listing 17: Delegates used by the Super Ability system (source: own elaboration)

When the Combat System performs a specific action, i.e. enables the Super Ability, an appropriate Delegate is called. UI binds an event to this Delegate which can be seen in figure 32.

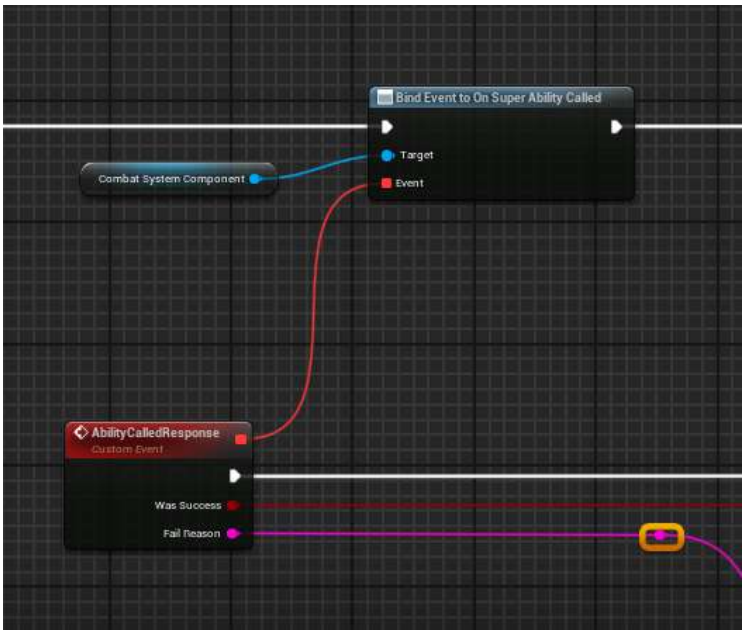

Fig. 32: Event binding in Combat UI(source: own elaboration)

5.3.8 Perfect Parry Implementation

Ability to deflect any enemy attack was also introduced to give players more options of dealing with adversaries during combat encounters. However, to increase difficulty, parry has to be perfect, meaning it has to be performed in the right moment.
To enter the parry state, Right Mouse Button has to be pressed. The first step was to add a parry window to every enemy animation, example of that can be seen in figure 33. If the player did not enter the parry state during that window, the enemy applies the regular damage.

Fig. 33: Perfect Parry time window(source: own elaboration)

However, if the player manages to parry in that correct moment, the enemy attack is interrupted and the player's parry response function is called, which implementation can be seen in Listing 18. Optionally, to aid players even further, for a brief moment a slow-motion can be played, by interpolating a curve via the FTimeline.

```
void UCombatSystemComponent::PerfectParryResponse(bool bEnableSlowMo)
{
  if (bEnableSlowMo)
  {
    bShouldSpeedUpSlowMoTimeline = false;
    ParrySlowMoTimeline.PlayFromStart();
  }

  AnimInstance->Montage_Play(ParryImpactMontage, ParryImpactMontageSpeed);

  NextAttackData = DetermineNextCounterAttackData();

  auto LaunchVelocity = playerCharacter->GetActorForwardVector() * -250.f;
  playerCharacter->LaunchCharacter(LaunchVelocity, false, false);

  PlayerCameraManager->StopAllCameraShakes();
  PlayerCameraManager->PlayWorldCameraShake(GetWorld(),
    ParryCameraShake,
    playerCharacter->GetActorLocation(),
    0, 500, 1);
}
```

Listing 18: Perfect Parry response function (source: own elaboration)

Unfortunately, the FTimeline (unlike its Blueprint counterpart) is affected by the Global Time Dilation needed to slow down the time. Leaving this issue unaddressed, would distort the actual time specified for the slow-motion.

To solve this, the FTimeline's playrate is adjusted in real-time with the following equation $\frac{1.0}{CurrentTimeDilation}$.

5.4 Combo system

The Combo System is a gameplay mechanic created to enhance replayability value of the game by awarding points to the player. The score achieved can be compared to other competitors globally, promoting rivalry among them (see 5.8.3). Points are allocated based on multiple kills in a row or performing custom actions such as "Wall-run Kill!". It also updates the user interface dynamically reflecting the previously mentioned actions through the use of Observer Pattern. The importance of the system has increased even further after making player's special abilities dependent on the number of Combo Points.

5.4.1 System Architecture

The system is implemented in the UComboClass, which inherits from Unreal Engine's UObject and serves as a primary manager class, which is responsible for all combo-related logic. Additionally, there is ComboTimerComponent which is attached to the player character and its role is crucial as it is managing all timers regarding Combo Levels and Kill Streaks. Both classes are structured in order to ensure flawless combo management and reward allocation without delay.

The system is based on the Singleton Pattern. The authors decided that this approach is optimal, as it guarantees that only a single global instance of the system is accessible across the game. In addition, it is memory-friendly to prevent unnecessary instances from being created. The GetInstance function is responsible for creating or returning the reference to the ComboSystem singleton object and can be seen in the listing 19.

```
UComboSystem* UComboSystem::GetInstance()
{
  if (Instance == nullptr)
  {
    Instance = NewObject<UComboSystem>();
    Instance->AddToRoot();
  }
  return Instance;
}
```

Listing 19: Instance creation using GetInstance function (source: own elaboration)

The Observer Pattern was used to ensure that all the widgets are dynamically updated whenever combo increases, or custom action is performed. Whenever the WComboSystem widget is notified, the display is updated to reflect the new combo status.

5.4.2 Core Functionality

The system's core functionality is to track players kill streaks and allocate adequate points to the player. First, whenever the enemy is killed, the IncreaseKillCount function is called (see listing 20). It increments the player's KillCount and triggers updates to the kill streak and combo level. Then, the notification is broadcasted to all observers using the KillIncreasedEvent delegate and starts the ComboTimer, which lasts 5 seconds.

```
void UComboSystem::IncreaseKillCount()
{
  if (!this)
  {
    return;
  }

  KillCount++;
```

```
  StartKillStreak();
  UpdateComboLevel();
  KillIncreasedEvent.Broadcast();
}
```

Listing 20: Increasing players kill count using IncreaseKillCount function (source: own elaboration)

Subsequently, the StartKillStreak function is called, which assigns descriptive labels to the user interface, for example, "Double Kill!", depending on the length of the kill streak. This function can be seen in listing 21. It also starts the KillStreakTimer, which lasts for 2 seconds, meaning that the player has a very limited time to perform consecutive kills. Additionally, it executes the HandleComboState function, which manages custom combo states, such as WallJump-Kill, and awards the player with additional points. Those custom actions are defined in EComboState enumeration.

```
void UComboSystem::StartKillStreak()
{
  HandleComboState();

  if (this->ComboState == EComboState::None)
  {
    KillStreakCount = (KillStreakCount > 0) ? KillStreakCount + 1 : 1;
  }
  else
  {
    ResetComboState();
  }

  int32 Points = 0;
  FString StreakName;

  switch (KillStreakCount)
  {
  case 2:
    Points = 1000;
    StreakName = "Double Kill!";
    break;
  case 3:
    Points = 2000;
    StreakName = "Triple Kill!";
    break;
  case 4:
    Points = 3000;
    StreakName = "Quadra Kill!";
    break;
```

```
  default:
    if (KillStreakCount >= 5)
    {
      Points = 5000;
      StreakName = "Killing Machine!";
    }
    break;
  }

  if (!StreakName.IsEmpty())
  {
    CurrentComboPoints += Points;
    KillStreakName = StreakName + FString::Printf(TEXT(" +%d"), Points);
    KillStreakMessages.Add(KillStreakName);
    OnNewKillStreakMessage.Broadcast(KillStreakName);
  }
}
```

Listing 21: Starting and increasing players kill count using StartKillStreak function (source: own elaboration)

The UpdateComboLevel function scales the combo points multiplier whenever a player's performance improves (see listing 22).

```
void UComboSystem::UpdateComboLevel()
{
  if (KillCount != PreviousKillCount)
  {
    ComboLevel++;
    float Multiplier = FMath::Pow(2.0f, static_cast<float>(ComboLevel));
    CurrentComboPoints += Multiplier * 100;
    PreviousKillCount = KillCount;
    OnComboStart.Broadcast();
  }
}
```

Listing 22: Scaling points multiplier using UpdateComboLevel function (source: own elaboration)

The player's performance is tied to Combo Level and is calculated based on the table 1:

Similarly to other functions, it broadcasts updates to observers which reflects changes in the user interface.

Whenever the ComboTimer or KillStreakTimer expires, the ResetCombo or EndKillStreak function is triggered. These functions ensure proper state management when a combo or kill streak ends. Earned points are added to total combo points, and all related variables and timers are reset. As always, relevant delegates are triggered

Table 1: Combo multiplier based on the level (source: own elaboration)

Combo Level	Multiplier	Points Awarded
1	2	200
2	4	400
3	8	800
n	2^n	100×2^n

to notify observers of the user interface update. Both functions can be seen in the listing 23.

```
void UComboSystem::EndKillStreak()
{
  if (AbilityComboPoints <= SuperAbilityCost)
  {
    AbilityComboPoints += CurrentComboPoints;

    if (AbilityComboPoints > SuperAbilityCost)
    {
      AbilityComboPoints = SuperAbilityCost;
    }
  }
  KillStreakCount = 0;
  OnResetKillstreak.Broadcast();
  KillStreakMessages.Empty();
}

void UComboSystem::ResetCombo()
{
  TotalComboPoints += CurrentComboPoints;
  CurrentComboPoints = 0;
  ComboLevel = 0;
  OnResetCombo.Broadcast();
  ResetComboState();
}
```

Listing 23: Ending Combo and KillStreak using EndKillStreak and ResetCombo functions (source: own elaboration)

5.4.3 Integration

The Combo System is connected to the players special abilities, such as Time Stop and Super Ability. Both require gathering Combo-Points up to a threshold to activate them. This feature was developed

for more strategic gameplay, and thus, it will demand close attention from players to ComboPoints to turn those abilities on when needed as they can change the outcome of the encounter.

5.4.4 Potential Future Development

The way Combo System is designed makes it highly adaptable. For example EComboState enumeration allows new combo states to be added with minimal changes. Furthermore, the system's singleton pattern, implemented in GetInstance function, ensures consistent access to the system across the game. By integrating Observer Pattern as well, the system performs real-time updates reflected in the user interface while maintaining modularity, creating a truly solid foundation for future development.

5.5 Enemy system

The purpose of this section is to provide a detailed explanation of the mechanics behind the AI Enemy. This system plays a crucial role in shaping the fast-paced and challenging nature of the game, making it a cornerstone of the overall gameplay experience.

Enemies detect the player in a specific range and view. Artificial intelligence (AI) is used to make game harder and more realistic. The engine supports two types of enemies: melee enemies and ranged enemies. Both types have different behaviour patterns and ways of attacking.

5.5.1 Melee Enemies

These are close enemies (see figure 34) that will actively pursue the player once they come within the detection range of two senses - vision, and hearing. Both senses are set in the characters' blueprints using PawnSensing Component. They bridge the gap between the player and their character with great speed and agility. In their design, melee enemies are meant to test the player's avoidance and defence capabilities, which will promote accurate timing and movement.

Fig. 34: Melee Enemy (source: `https://paragon.fandom.com/wiki/Kallari`)

Melee enemies can also be blocked, giving the player blocking action against them. The parrying starts when the player successfully presses LMB during the attack, indicated by the enemy's knife glowing yellow or red. It momentarily stuns the enemy and presents an opening for quick killing Not only are the blueprints used in this process, but animations also play an important role.

There is also a second type of enemy. They differ from the prior melee enemies in both appearance (see figure 35) and the special ability they own - teleportation. The teleportation works in such a way that, as the enemy gets closer to the player, it teleports to them at super speed. During this teleportation, the enemy vanishes for a moment, leaving behind a trail of special purple smoke. It makes player to stop it with their parry ability before attacking. The teleport ability is used by the enemy only once within a long period.

Fig. 35: Melee Enemy 2. Type (source: own elaboration)

The teleportation ability (see figure 36) requires significant changes in typical scenarios for melee enemies. Time, player range, and location must be calculated with the best possible approximations. The key components are included in the behaviour tree through specially created decorators, tasks, and additional blackboard variables.

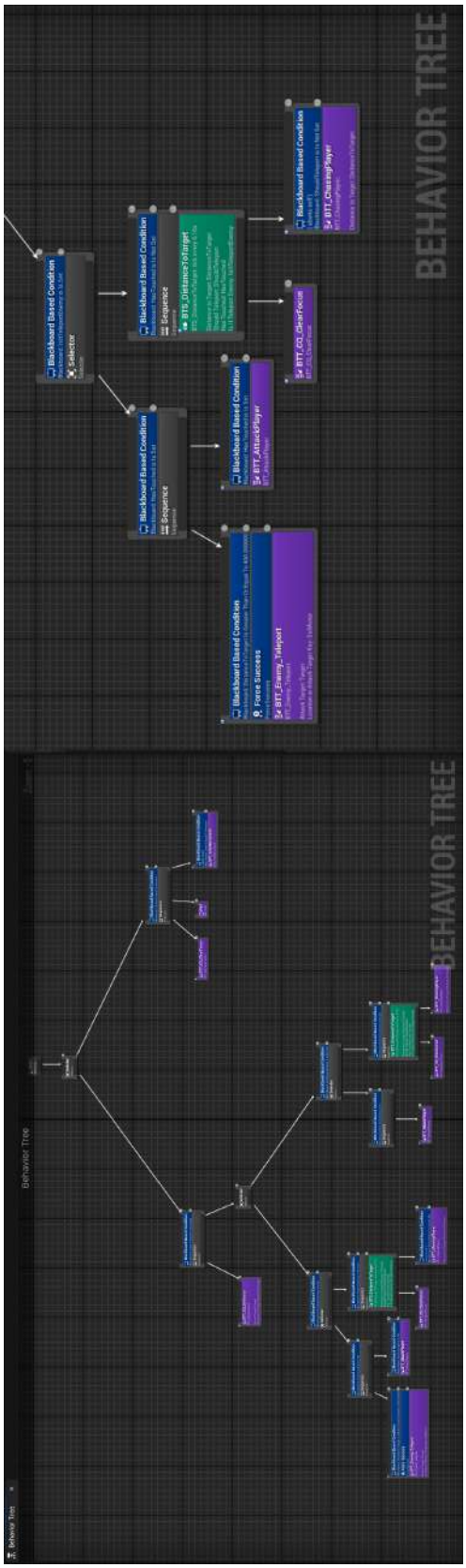

Fig. 36: Melee Enemy's behaviour tree (source: own elaboration)

5.5.2 Ranged Enemies

These are enemies (see figure 37) that attack players from a distance, using projectiles to apply pressure to the player without necessarily closing the gap.

Fig. 37: Ranged Enemy (source: `https://paragon.fandom.com/wiki/Wraith?file=Wraith_Lunar_Ops_skin.jpg#Tier_I_Skins_`)

Besides senses like those in melee enemies, their movement is defined in the previously mentioned AI Asset - Environment Query (see figure 38). They have several conditions to remain in position relative to the player: having a clear shot, not being attacked, and not being too far away to shoot effectively.

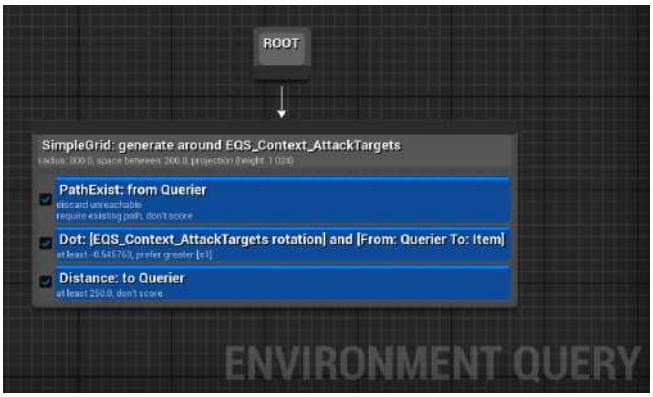

Fig. 38: Ranged Enemy's Environment Query (source: own elaboration)

Each projectile (see figure 39) has an extra box collider, so the player could deflect bullets back and kill the enemy with them. This process mimics the parrying mechanic used for melee enemies. Parrying is possible when the bullet enters an acceptable range of the player and glows red or yellow.

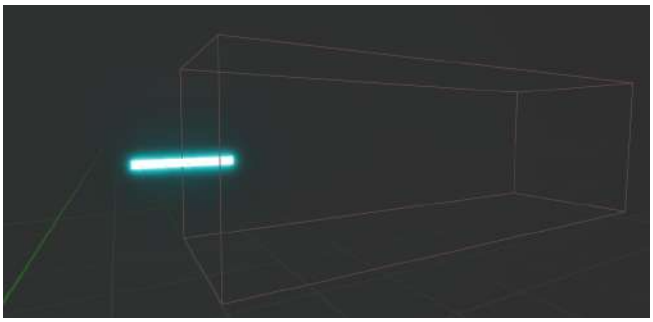

Fig. 39: Ranged Enemy's Projectile (source: own elaboration)

To achieve the best performance, projectiles are managed using an object pooling pattern. It is implemented using two main blueprints, BP_BulletPool and BP_Bullet, that interact with each other. The pool is placed far away from the main playing field because complexity of the ranged enemy system makes it impractical

for the pool to be owned by the enemy itself. Projectiles get teleported to the barrel of the gun of the enemy at the right time and shoot forward along a specified vector. There are three scenarios that determine what happens next with a bullet:

1. **The bullet hits the player:**
   - If the player is hit, they die.
   - If the player parries the bullet, its direction changes, aiming at the ranged enemy. If the enemy is hit, they die.
2. **The bullet hits static object or any enemy:**
   - If it collides with a static object, the object pooling system is triggered.
   - If it accidentally hits an enemy, the enemy is killed.
3. **The bullet doesn't hit anything:**
   - After a set time, the object pooling system is triggered.

In each of these scenarios, the object pooling system ensures that the bullet is returned to the pool, ready to be reused.
In addition, some of the ranged enemies are equipped with shields (see figure 40), which they keep close to their bodies to protect themselves from attacks. It is implemented as an additional component in the character's blueprint. This can make these enemies more challenging to defeat for the player. In this case, there is the possibility that it can give the player reasons to take advantage of other ingame mechanics, such as the special ability or the bullet deflection feature, to brake the shield and kill them.

Fig. 40: Ranged Enemy's shield (source: own elaboration))

5.6 Time Manipulation System

The Time Manipulation System is designed to control the flow of time, providing the player with two powerful features. First, Time Stop ability, stops all enemies in place, providing a chance to strike without repercussions. The Rewind Time ability, on the other hand, allows the player to turn back time for a specified duration after death, however it can be used only once per level, requiring the user to consider its use carefully. The system was developed based on Marcus Reid's Unreal Engine 5 Rewind prototype, which is available under MIT licence here [15]. Even though, Reid's prototype laid the groundwork for the system, the authors expanded it with additional features such as player safety checks, rewinding for a specified

duration, post-processing effect, user interface elements and other optimizations and modifications that are unique to the project.

5.6.1 Architecture and Core Functionality

The rewind system is implemented as an actor component called RewindComponent, which allows it to be attached to any actor. Its core functionality consists of capturing, managing, and interpolating snapshots of actor states. These snapshots store those states at various points in time. A singular snapshot includes these variables:

- Transform (Actor's position, rotation and scale)
- LinearVelocity (Movement speed and direction)
- AngularVelocity (Rational speed and direction)
- TimeSinceLastSnapshot (Elapsed time since last snapshot)
- bIsLocationSafe (Boolean indicating if location is safe)

Snapshots are captured in the Tick event and stored in the Ring Buffer data structure called TransformAndVelocitySnapshots. A Ring Buffer, also known as Circular Buffer allows for overwriting old snapshots when the structure becomes full, which is efficient for streams of data like that. More detailed description of what exactly Ring Buffer is can be found here[6].

5.6.2 The Process of Rewinding Time

The process of rewinding time starts with HandleInsufficientSnapshots function that ensures enough snapshots exist. It validates their consistency which prevents operations on invalid snapshots (see listing 24). This step can prevent crashes or unintended behaviour when snapshot array is uninitialized or empty.

```
1 bool URewindComponent::HandleInsufficientSnapshots()
2 {
3   check(!bSnapshotMovementVelocityAndMode || TransformAndVelocitySnapshots
      .Num() == MovementVelocityAndModeSnapshots.Num());
4
5   if (LatestSnapshotIndex < 0 || TransformAndVelocitySnapshots.Num() == 0)
6   {
7     return true;
8   }
```

```
9
10    if (TransformAndVelocitySnapshots.Num() == 1)
11    {
12      ApplySnapshot(TransformAndVelocitySnapshots[0], false);
13      if (bSnapshotMovementVelocityAndMode)
14      {
15        ApplySnapshot(MovementVelocityAndModeSnapshots[0], true);
16      }
17      return true;
18    }
19
20    check(LatestSnapshotIndex >= 0 && LatestSnapshotIndex <
        TransformAndVelocitySnapshots.Num());
21    return false;
22  }
```

Listing 24: Using HandleInsufficientSnapshots function to prevent operations on invalid snapshots (source: Original code from Marcus Reid, modified and adapted by the authors)

When validation passes, the interpolated state is calculated by blending two snapshots based on InterpolationFactor. It represents how far along the timeline the two snapshots have progressed. This factor is implemented in the InterpolateAndApplySnapshots function and can be seen in the listing 25.

```
1  void URewindComponent::InterpolateAndApplySnapshots(bool bRewinding)
2  {
3    constexpr int MinSnapshotsForInterpolation = 2;
4    check(TransformAndVelocitySnapshots.Num() >=
       MinSnapshotsForInterpolation);
5    check((bRewinding && LatestSnapshotIndex < TransformAndVelocitySnapshots
       .Num() - 1) ||
6        (!bRewinding && LatestSnapshotIndex > 0));
7
8    int PreviousIndex = bRewinding ? LatestSnapshotIndex + 1 :
       LatestSnapshotIndex - 1;
9    float InterpolationFactor = TimeSinceSnapshotsChanged /
       TransformAndVelocitySnapshots[LatestSnapshotIndex].
       TimeSinceLastSnapshot;
10
11   const FTransformAndVelocitySnapshot& PreviousSnapshot =
       TransformAndVelocitySnapshots[PreviousIndex];
12   const FTransformAndVelocitySnapshot& NextSnapshot =
       TransformAndVelocitySnapshots[LatestSnapshotIndex];
13   ApplySnapshot(BlendSnapshots(PreviousSnapshot, NextSnapshot,
       InterpolationFactor), false);
14
15   if (bSnapshotMovementVelocityAndMode)
```

```
16   {
17     const FMovementVelocityAndModeSnapshot& PreviousMovementSnapshot =
       MovementVelocityAndModeSnapshots[PreviousIndex];
18     const FMovementVelocityAndModeSnapshot& NextMovementSnapshot =
       MovementVelocityAndModeSnapshots[LatestSnapshotIndex];
19     ApplySnapshot(BlendSnapshots(PreviousMovementSnapshot,
       NextMovementSnapshot, InterpolationFactor), true);
20   }
21 }
```

Listing 25: Interpolating snapshots using InterpolateAndApplySnapshots function (source: Original code from Marcus Reid, modified and adapted by the authors)

The BlendSnapshots function ensures smooth transitions between states, avoiding sudden jumps in position, rotation, or velocity (see listing 26).

```
1  FTransformAndVelocitySnapshot URewindComponent::BlendSnapshots(
2    const FTransformAndVelocitySnapshot& A,
3    const FTransformAndVelocitySnapshot& B,
4    float Alpha)
5  {
6    Alpha = FMath::Clamp(Alpha, 0.0f, 1.0f);
7    FTransformAndVelocitySnapshot BlendedSnapshot;
8    BlendedSnapshot.Transform.Blend(A.Transform, B.Transform, Alpha);
9    BlendedSnapshot.LinearVelocity = FMath::Lerp(A.LinearVelocity, B.
       LinearVelocity, Alpha);
10   BlendedSnapshot.AngularVelocityInRadians = FMath::Lerp(A.
       AngularVelocityInRadians, B.AngularVelocityInRadians, Alpha);
11   return BlendedSnapshot;
12 }
```

Listing 26: Ensuring smooth transitions using BlendSnapshots function (source: Original code from Marcus Reid, modified and adapted by the authors)

After blending, the interpolated state is finally applied to the actor. The ApplySnapshot function updates the transform and velocities of the actors and can be seen in the listing 27.

```
1  void URewindComponent::ApplySnapshot(const FTransformAndVelocitySnapshot&
       Snapshot, bool bApplyPhysics)
2  {
3    GetOwner()->SetActorTransform(Snapshot.Transform);
4    if (OwnerRootComponent && bApplyPhysics)
5    {
6      OwnerRootComponent->SetPhysicsLinearVelocity(Snapshot.LinearVelocity);
```

```
7       OwnerRootComponent->SetPhysicsAngularVelocityInRadians(Snapshot.
        AngularVelocityInRadians);
8     }
9   }
```

Listing 27: Applying snapshots using ApplySnapshot function (source: Original code from Marcus Reid, modified and adapted by the authors)

5.6.3 Other Functionalities

From a user's perspective, one of the core functionalities is Rewind for duration, as it is the feature that lets the player rewind time after death. The function responsible for that is RewindForDuration, which takes duration on input (see listing 28). Following that, the rewind process starts until the time runs out.

```
1  void URewindComponent::RewindForDuration(float Duration)
2  {
3    if (!GameMode)
4    {
5      return;
6    }
7    GameMode->SetRewindSpeedFastest();
8    RemainingRewindDuration = Duration;
9    bIsRewindingForDuration = true;
10   OnGlobalRewindStarted();
11 }
```

Listing 28: Rewinding time for specified duration using RewindForDuration function (source: own elaboration)

Time Stop has a similar function called TimeScrubForDuration. It works similarly to RewindForDuration but serves as an ability for the player to stop time, by stopping all actors except the player in the latest snapshot (see listing 29).

```
1  void URewindComponent::TimeScrubForDuration(float Duration)
2  {
3    if (!GameMode)
4    {
5      return;
6    }
7    TotalTimeScrub = Duration;
8    TimeScrubStartedAt = GetWorld()->GetTimeSeconds();
9    bIsTimeScrubbingForDuration = true;
```

```
10    GameMode->ToggleTimeScrub();
11    GetWorld()->GetTimerManager().SetTimer(RewindTimerHandle, this, &
        URewindComponent::StopTimeScrubForDuration, Duration, false);
12  }
```

Listing 29: Stopping time for specified duration using TimeScrubForDuration function (source: own elaboration)

Additionally, the system uses a safety function called PerformSafetyTrace, which checks if the last location after the RewindForDuration has a surface below the player that he can safely stand on to prevent a meaningless death. Otherwise, if such a surface is absent, the rewind continues until it finds that location. It is done by creating a trace pointing below the player that checks if there is any object within the range below, that the player can stand on. The function can be seen in the listing 30.

```
1  bool URewindComponent::PerformSafetyTrace(const FVector& Location) const
2  {
3    FVector Start = Location;
4    FVector End = Location - FVector(0, 0, 150.0f);
5
6    FHitResult HitResult;
7    FCollisionQueryParams CollisionParams;
8    CollisionParams.AddIgnoredActor(GetOwner());
9
10   bool bHit = GetWorld()->LineTraceSingleByChannel(HitResult, Start, End,
       ECC_Visibility, CollisionParams);
11
12   return bHit;
13 }
```

Listing 30: Checking safety of players postion using PerformSafetyTrace function (source: own elaboration)

5.6.4 Post-processing Effect

The post-processing effect was created to signal the start and stop of time manipulation-related abilities visually. At the core of the effect is a radial gradient mask starting as a small circle in the middle of the screen, which gradually expands outward to cover the whole viewport. A panner node is used to slightly distort the gradient as it expands, producing a warping effect. The progress of the

circle is controlled by the scalar parameter. While the circle expands, the circular mask is used to blend normal screen textures with desaturated screen, creating the perception that the effect is spreading from the centre of screen (see figure 41).

Fig. 41: Distorted circle at the start of the effect (source: own elaboration)

The effect gradually desaturates the entire scene, leaving only black and white colors. While the effect is still on, all the enemies are highlighted in red. This is achieved by stencil buffers, which apply a mask to the enemies based on their stencil ID, replacing their appearance with a red overlay (see figure 42).

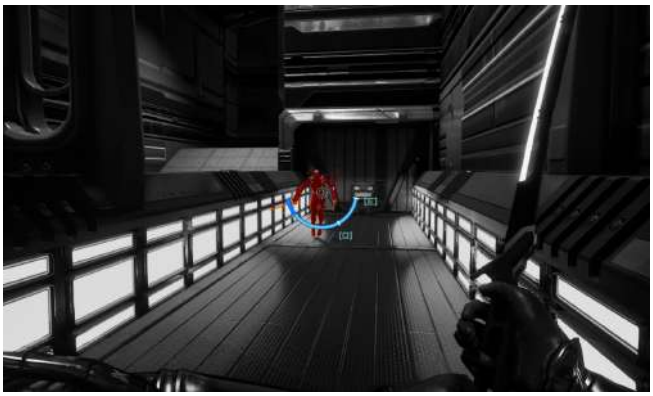

Fig. 42: Desaturated screen and enemies with applied mask (source: own elaboration)

5.7 Gameplay

Similarly to Ghostrunner, the main purpose of the gameplay is for players to learn from their mistakes and reapply that knowledge during another retry. It is rather difficult to achieve, as there are many factors to consider, but after the careful analysis, the authors were able to come up with the following game loop presented in the figure 43.

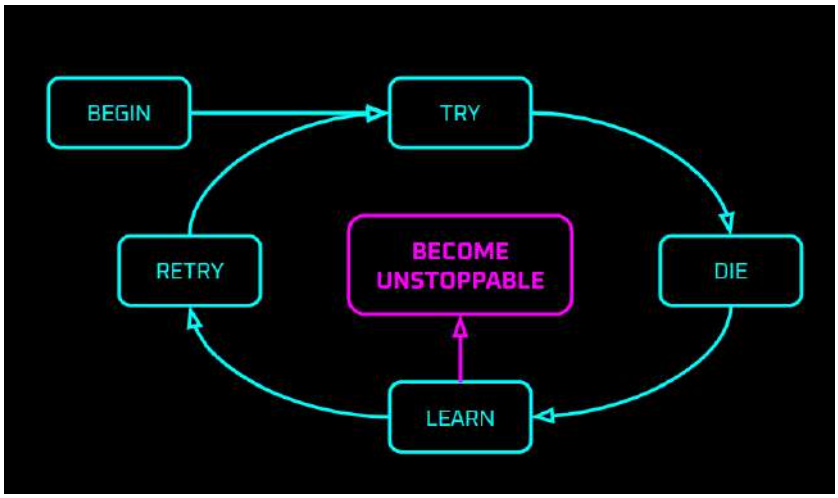

Fig. 43: The Neon Shadows gameplay loop (source: own elaboration)

Combining rapid and responsive parkour movement, brutal combat with challenging enemies helped bring the authors vision to life and make the game demanding enough so the gameplay loop presented above is required path the to player's success.

5.8 Statistics Save System

The Save System is a key component in every game in the current market and as the project grew larger, authors realized that it is indeed crucial to ensure that player's progression persists across gameplay sessions. The system not only saves and loads data, but also allows for Steam profile integration, unlocking new levels, and even sending data to online leaderboards.

5.8.1 Core Functionality

Saving and loading data is the primary feature of the system, therefore ensuring that player progress is securely and uniquely saved is a must. This is achieved by using player's unique Steam ID as an identifier, which serves as the basis for generating names for save slots specific to each player. As a result it is possible to have multiple users with separate progression on the same device. However, if Steam is not connected, the system creates a local save file which is lacking online leaderboards feature and can be overwritten by any player using same device.

This is achieved by the GetSlotName function which returns a save slot based on player's Steam ID. If there is no Steam connection, the system defaults to local save (see listing 31).

```
FString ULevelSummaryData::GetSlotName() const
{
    return SaveGameInstance && !SaveGameInstance->SteamID.IsEmpty()
        ? SaveGameInstance->SteamID + "_LevelSummary"
        : "Local_LevelSummary";
}
```

Listing 31: Getting the save slot using GetSlotName function (source: own elaboration)

Steam credentials are gathered at the start of the program using GatherSteamData function (see listing 32).The actual process of getting them is straightforward due to Advanced Session plugin, which

for example changes the behaviour of GetUniqueNetID node to getting Steam ID. This approach allows for accessing and displaying them in menus, leaderboards and save summaries.

```
void ULevelSummaryData::GatherSteamData(FString SteamName, FString SteamID
    )
{
    if (SaveGameInstance)
    {
        SaveGameInstance->SteamID = SteamID;
        SaveGameInstance->SteamUsername = SteamName;
    }
}
```

Listing 32: Getting Steam credentials using GatherSteamData function (source: own elaboration)

Saving is possible due to custom ULevelSummarySaveGame class that inherits from Unreal Engine's USaveGame class and acts as the primary container for all saved data. Fields of each save file include:

- `TArray<FLevelData> LevelSummaries` - Array of `FLevelData` structures that store level-specific stats such as: Level Name, Combo Points, Player Deaths, Elapsed Time, and Total Score.
- `TArray<FString> UnlockedLevels` - Array consisting of unlocked level names represented by strings.
- `FString SteamID` and `SteamUsername` - Previously mentioned Steam credentials.

The process of actual file writing starts with AddLevelData function which populates FLevelData structure with all the needed values (see listing 33). This action is performed at the end of the level when summary screen widget is constructed. Subsequently, SaveSummaryDataToSlot function is executed which writes save file to disk using Unreal's Engine SaveGameToSlot method. It is worth mentioning that the new TotalScore variable is compared with the one already saved, so if its lower, the function will not be carried out.

```
bool ULevelSummaryData::AddLevelData(const FLevelData& LevelData)
{
    if (!SaveGameInstance)
    {
        return false;
    }
    for (FLevelData& ExistingData : SaveGameInstance->LevelSummaries)
```

```
    {
        if (ExistingData.LevelName == LevelData.LevelName)
        {
            if (IsNewLevelScoreHigher(LevelData))
            {
                ExistingData = LevelData;
                return true;
            }
            return false;
        }
    }
    SaveGameInstance->LevelSummaries.Add(LevelData);
    return true;
}
```

Listing 33: Populate FLevelData structure using AddLevelData function (source: own elaboration)

Regarding loading data, the LoadSavedData function is executed at the start of the program. Similarly to saving data it uses Unreal Engine's LoadGameFromSlot method. After data is loaded, it is possible for the player to view it in an appropriate menu due to it being accessible at any point during the game.

5.8.2 Level Unlocking

The progression system heavily relies on the ability to unlock levels dynamically each time player beats a new level. It is implemented through a previously mentioned UnlockedLevels string array in the save system. To unlock a new level, UnlockLevel function is utilized, which adds a string with next level name to the array (see listing 34). Similarly to other save system features, its executed when the Summary Widget is constructed. This implementation ensures that the player can access only unlocked levels, by enabling buttons in Level Selection screen corresponding to level names present in the array.

```
bool ULevelSummaryData::UnlockLevel(const FString& LevelName)
{
    FRegexPattern LevelPattern(TEXT("^Level\\d+$"));
    FRegexMatcher LevelMatcher(LevelPattern, LevelName);

    if (!LevelMatcher.FindNext())
    {
```

```
8           return false;
9       }
10
11      if (!SaveGameInstance->UnlockedLevels.Contains(LevelName))
12      {
13          SaveGameInstance->UnlockedLevels.Add(LevelName);
14          return true;
15      }
16      return false;
17  }
```

Listing 34: Unlocking level using UnlockLevel function (source: own elaboration)

5.8.3 Online Leaderboards

Building upon the saved player data and level progression, the game integrates an online leaderboard system that allows players to compare their performance against others globally. This feature was implemented by using Epic Leaderboard plugin, which allows for quick online leaderboard implementation in any project. The plugin only allows for two variables which are Score and Name. Unfortunately, the Name field must be unique because it serves as an identification of an entry for the plugin. This presented a challenge, because Steam allows multiple users to have identical usernames. To solve this problem, the authors decided to merge player's Steam username and ID to a single string separated by "@". The "@" symbol was chosen due to it being one of the forbidden symbols in Steam usernames, ensuring no conflicts arise. For example player with username "SteamUser" and ID "76561198087248360" would be represented by "SteamUser@76561198087248360" in the leaderboard database. Of course, in the graphical representation of the leaderboard in the game, only the Username part is visible to the player, omitting the "@SteamID" for cleaner and more user-friendly interface. As for the Score field, its represented by TotalScore variable without any modifications, that is already present in the save system.

The crucial aspect of the leaderboard implementation was determining when the data is sent to the online leaderboard. To ensure relevant updates the entry is submitted each time the player beats

the level. Naturally, before submission, the score is compared to the existing entry to determine if it surpasses the previous value.

5.9 User Interface

The User Interface (UI) in The Neon Shadows was designed to ensure players can access all the essential gameplay functions without distraction, maintaining a sleek and futuristic appearance. The UI design employs a blue toned colour palette, highlighting active buttons and checkboxes.

5.9.1 Menus

The UI features several key menus, each designed to provide intuitive navigation and functionality:

1. **Main menu**
   This is where the adventure begins. The main menu introduces players to the game's theme and provides access to all other sections (see figure 44).

Fig. 44: The main menu page (source: own elaboration)

2. **Pause menu**
   Accessible at any time, the pause menu allows players to adjust

settings, leave the game, or restart the current level (see figure 45).

Fig. 45: The pause menu (source: own elaboration)

3. **Settings**
   The settings menu featuring all the settings in 3 categories: Game, Audio and Video (see figure 46).

   – **Game**: Gameplay settings such as controls and displaying information.

   – **Audio**: Volume adjustments for music and sound effects.

   – **Video**: Graphics settings, including resolution, quality, and display modes.

Fig. 46: Settings menu (source: own elaboration)

4. **Leaderboards**
   Displaying a scoreboard with results for each level, encouraging competition and replayability (see figure 47).

Fig. 47: Leaderboards (source: own elaboration)

5. **Level selection**
   The menu that allows to select a level to pass (see figure 48).

Screenshots of every level added on top of the Level component give a distinguishable overlook to what that level is featuring.

Fig. 48: Level selection menu (source: own elaboration)

Each of those pages was created using consistent UI components such as:

- **Buttons** - designed in three variations for normal, hovered, and clicked states.
- **Sliders** - used for adjustable settings such as volume and mouse sensitivity.
- **Checkboxes** - used for switchable options like vertical synchronisation or dynamic resolution.

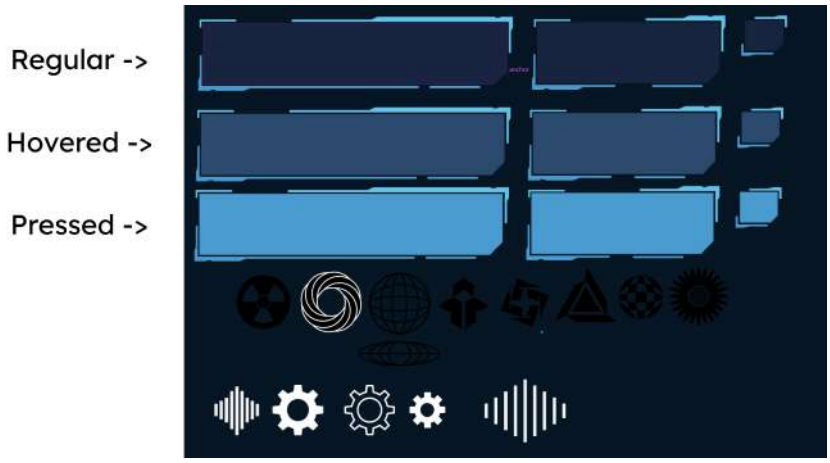

Fig. 49: User interface components created in Adobe Illustrator (source: own elaboration)

These components were designed in Adobe Illustrator (see figure 49) and then exported and implemented in Unreal Engine. This created an easy workflow and an option to change the colour palette or styling inside the component, to apply changes on all project pages. The UI interacts with statistics save system and Steam, preserving player preferences and progress.

5.9.2 Heads Up Display

In a game that focuses on speed and agility, nothing should obscure player's field of view. It was an important requirement to fulfil when designing and implementing the Heads Up Display (HUD) in the game described. It was designed as a transparent Neuralink interface, focusing on clarity and usability by presenting only the most relevant information to the player at any given moment, while immersing players into the lore even further.
The full player's in-game UI is presented in the figure 50 below.

Fig. 50: Player's HUD (source: own elaboration)

Collecting the Combo Points is a crucial mechanic, that allows players to activate special abilities, thus it was decided that the current Combo Points Meter should be constantly present in the center of the screen. This way, even during the most chaotic combat encounters, players can quickly find out how many points they have earned. It was implemented as a Radial Bar, which fill amount is dictated by the following equation:

$$Fill = 1 - \frac{CurrentComboPoints}{MaxComboPoints}$$

The complement to one is required, as authors wanted the radial bar to be filled in the left-to-right direction, which by default is reversed.

As the amount of the Combo Points controls the activation of the special abilities, the two icons corresponding to their key-bindings were added. Once enough points are earned, the correct icon changes its opacity, indicating that this particular ability can be now activated. Combo Bar, along with the icons, is depicted on figure 51.

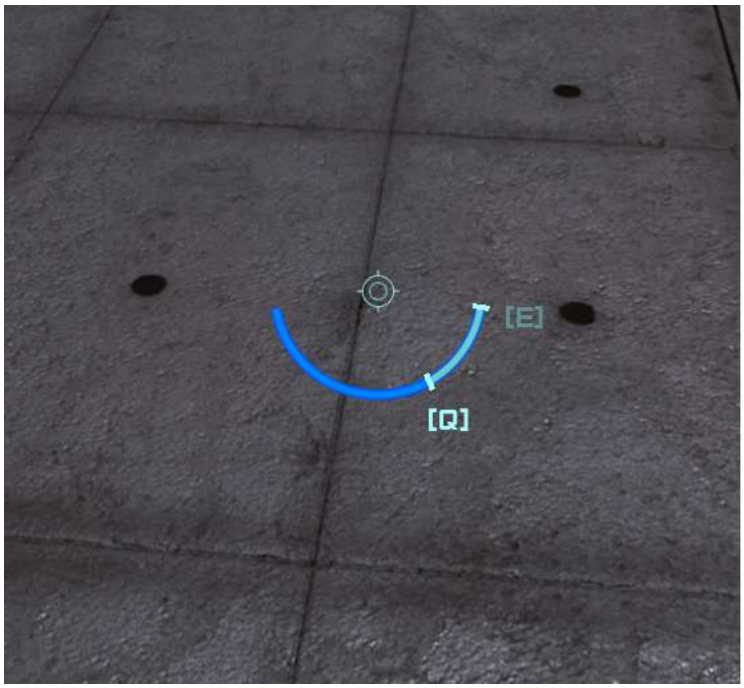

Fig. 51: Combo Bar (source: own elaboration)

The total amount of Combo Points is displayed in the top-left corner of the screen, and a crucial mechanic for speedrunning - the timer - is constantly present in the top-right, showing the player how much time did they spend on the current level so far.

To showcase the current state of the player's Combo, after an enemy was killed, an additional widget appears in the bottom-left of

the screen, that can be seen in figure 52. It updates based on how well the player performs during combat.

Fig. 52: Combo UI progression (source: own elaboration)

To ensure system decoupling and clean architecture an Observer Pattern, described in the detail in section 4.4, was used. Different widgets observe other systems (such as Combat System or Combo System) and execute the proper functionality by binding to the correct event.

5.10 Level Design

Level Design is one of the hardest aspects of making a game. Besides providing aesthetics and storytelling, level design plays a critical role in delivering an engaging gameplay that utilizes movement mechanics. Levels should be carefully balanced, to progressively increase difficulty with each section, challenging players while keeping the experience rewarding and satisfying. While the game follows a corridor-based structure, certain sections are designed with multiple paths to allow varied playstyles and encourage exploration. These alternative routes increase replayability and provide opportunities for players to experiment with different strategies, particularly for-speedrunning.

There are key factors that are persisting in every level created in The Neon Shadows discussed in the following subsections.

5.10.1 Aesthetics

Despite limited resources, the developers have created visually appealing and professional-looking levels, maintaining the immersive Cyberpunk aesthetic. The aesthetic in level design is reflected using futuristic architecture, objects, materials, level themes (like future factory) and dystopian atmosphere, carefully achieved using lighting, graffiti, destroyed facilities and color grading. Each space and object in the game world is designed with intention, meaning and purpose, ensuring that every element contributes to the overall immersion and narrative depth.

Fig. 53: Level 1 cargo storage and a broken staircase (source: own elaboration)

The game's aesthetic should evolve in line with the story progression, visually showcasing the player's journey fro the lowest city levels to the luxurious, oppressive domains of the ruling elite. Early levels feature environments inspired by futuristic warehouses, factories, cargo harbours, and abandoned metro line undergrounds, representing the hardships of the city's working class and remnants of an older way of life.

The tutorial level departs from this physical setting, adopting a virtual reality aesthetic. This design choice ties with the narrative, showing the protagonist's everyday training.

5.10.2 Clear goal

To maintain a consistent game flow and sustain player interest, the goal in each level must be easily identifiable. Each level in The Neon Shadows is designed with a straightforward objective: reaching the end and eliminating enemies on the way.

Fig. 54: Yellow projector lighting a wall (source: own elaboration)

Visual cues are guiding players through the environment. Elements such as lighting, environmental contrast, map layouts, recognizable objects, and purposeful use of colors ensure players can intuitively navigate their surroundings. In particular, the color yellow is established early in the game as a key indicator of the correct path forward, helping to create a visual language that players learn to rely on.

Fig. 55: Showcase of player attention getter (source: own elaboration)

Additional visual helpers, such as PostFX illusions, graffiti arrows, and text markings, are used to draw the player's attention and spark curiosity about what lies ahead. These elements not only guide players but also contribute to maintaining engaging flow throughout the level.

5.10.3 Traverse parkour

Each level in designed to maximize the use of advanced movement mechanics provided by the parkour system, encouraging players to utilize wall-running, sliding, double jumping, mantling and dashing seamlessly in their traversal. Specific objects, such as slopes, rounded walls and etcetera are intentionally placed to promote the use of parkour mechanics, providing players with opportunities to experiment with different movement options while navigating obstacles and fighting enemies.

Fig. 56: Level 2 start (source: own elaboration)

Environmental obstacles, such as barriers, lasers, hot walls and flames, heighten the challenge and introduce a new variable the players need to cope with while fighting with enemies. This obstacles demand strategic thinking and quick reflexes, adding an additional layer of complexity to traversal and combat.Level sections differ in terms of their layout, some are more open arias and some are close hallways with lots of enemies.This diversity ensures players are constantly adapting to new circumstances, making the gameplay more dynamic and engaging.

5.10.4 Difficulty

A well-designed game provides a clear sense of progression and a balance between challenge and reward, maintaining players engagement. In The Neon Shadows, difficulty is gradually increasing players advance through the levels ensuring they are consistently challenged.

Fig. 57: Enemy displacement (source: own elaboration)

Balanced enemy and obstacle placement is a key to design an environments that challenge the player without being frustrating or too easy. By making encounters increasingly difficult every player has a chance to master mechanics and find his challenge. Besides this, the growing difficulty mirrors the player's journey.

For those seeking an additional layer of difficulty, the speedrunning mechanics pose an engaging challenge, encouraging players to refine their skills, develop new strategies and push their limits. Shortcuts, alternate routes, and momentum-based challenges in combination with speedrun mechanics promote replayability.

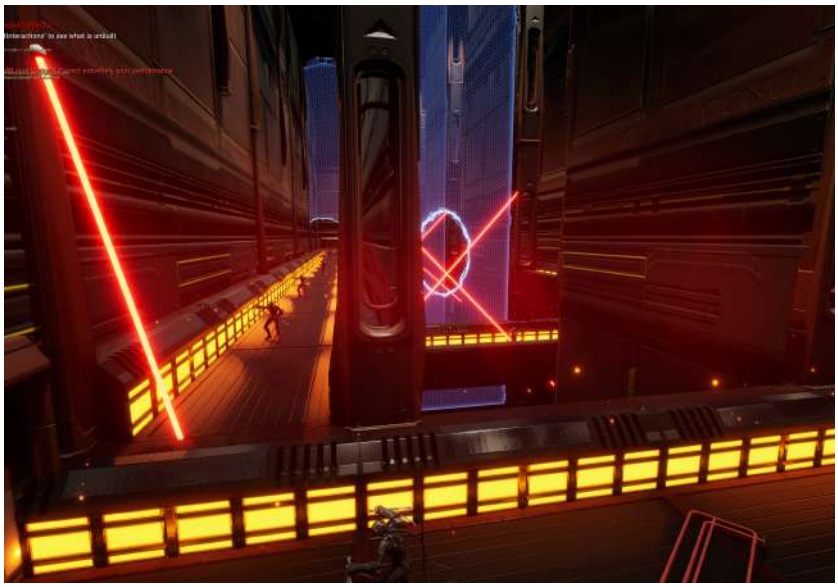

Fig. 58: Factory two way and a central shortcut (source: own elaboration)

5.10.5 Keeping it fresh

Each level in The Neon Shadows features a unique element that sets it apart, leaving a lasting impression. Destructible objects, distinctive layouts, lasers, conveyors, doors and other dynamic elements make each level interesting in a different way. This variety is keeping players attention and build anticipation for what surprises next level may hold, making the game exciting.

Additionally, making each encounter distinct not only adds variety but also increases the challenge, requiring players to continuously adapt to their surroundings and strategically utilize in-game mechanics. This balance between novelty and complexity keeps gameplay dynamic and rewarding.

Fig. 59: Underground metro (source: own elaboration)

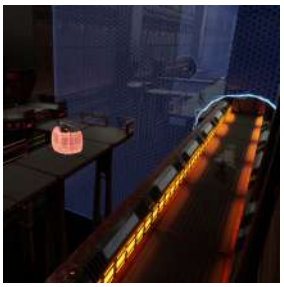

Fig. 60: Factory environment (source: own elaboration)

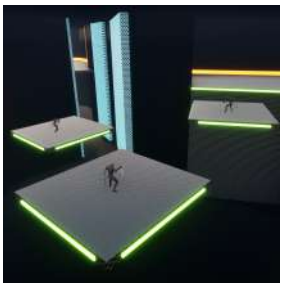

Fig. 61: Tutorial level (source: own elaboration)

5.11 Procedural Mesh Slicing

While designing the combat in the game described, authors wanted to give each kill a strong impact. It is partially achieved in the form of Camera Shakes - for the sword swing and hit reactions. To increase the viscerality of experience and player's satisfaction it was decided that also a procedural enemy Mesh slicing along the Katana's swing plane should be added.

5.11.1 Inspirations

Ghostrunner - the game that had the heaviest influence on the project design - features the enemy slicing. However, it is unlikely to be procedural.. Authors believe that there are couple of different Skeletal Meshes that were already cut. The game then checks the Katana slice plane once the enemy was hit and spawns the appropriate ones. It gives the illusion that the enemy was actually sliced along the current Katana trajectory.
The author's goal was more ambitious, resembling the system seen in *Metal Gear Rising Revengeance*, where any enemy can be cut any number of times along any slice plane, while retaining the ability to play an animation and simulate ragdoll. The following section shows the authors attempts to mimic such system and problems encountered along the way with the argumentation why the end system had to be simplified.

5.11.2 Challenges

There were a number of issues that had to be solved to make the system robust and visually pleasing:

1. In Unreal Engine, only the slicing of Static Meshes is supported. Skeletal Meshes by default cannot be cut, enforcing a fully custom solution.
2. The function mentioned in the previous point, splits any shape always into two parts. When cutting a human-like figure, greater precision is required as each cut debris should be treated separately.
3. The mesh should be sliced by an arbitrary plane. Most of the solutions gathered by the authors during the research phase featured separating the mesh between the bones, not an arbitrary point on the mesh.
4. Once the mesh is cut, it is required, that it can simulate ragdoll. Otherwise, the enemy's corpse will appear too rigid and cause unwanted and rather ridiculous scenarios.

5.11.3 Initial approaches

There were several approaches the authors tried to work with to make it satisfactory:

1. Procedural Mesh Conversion of Skeletal Mesh

   The first approach involved hiding the enemy's skeletal mesh upon slicing and spawning static mesh. This static mesh was then converted to a procedural mesh, allowing the authors to use Unreal Engine's Slice Procedural Mesh function. Unfortunately, this was not satisfying for various reasons:
   - The procedural mesh's nature made enemies appear unnaturally stiff after the slice. Combined with simple collision, this made sliced enemies clip through objects or block the player.
   - The caps generated on sliced enemies occasionally failed to render correctly, resulting in visual gaps in the cut areas.
   - When the enemy was sliced at certain angles, the engine misinterpreted the material mappings. For example, the texture

of the enemy's eye would cover his torso, making it completely gold.
- The Static to Procedural Mesh conversion process was costly, making a noticeable performance drop when slicing multiple enemies.

2. Use of Chaos Flesh Plugin

   The second approach involved the usage of Chaos Flesh Plugin, typically used for soft body physics or deformation on skeletal meshes. Given its ability to handle deformation, initially, it was a good candidate for implementing procedural slicing. However, this approach presented challenges impossible to overcome:
   - The plugin heavily relies on skeletal meshes, meaning that applying it only to static meshes makes it behave like a sponge.
   - The plugin is not optimized for real-time slicing, leading to implementation difficulties.

3. Bone Detachment

   The third approach involved detaching bones dynamically during slice. By removing the skeletal connection between sections of the enemy, the sliced parts could be separated in real time while maintaining the skeletal nature of the mesh. However, it introduced one crucial issue:
   - The detachment of bones always causes unnatural stretching, as if trying to connect those parts, making it visually unappealing.

5.11.4 Final solution

For the end solution, the authors limited the slicing system to the head bone.
It was noticed that the Katana tends to hit mainly the upper parts of the body - usually around the neck - thus this simplification seemed valid. It brought the best of both worlds - the simplicity and the

increase in the kill impact and satisfaction.
There are two distinct enemy Skeletal Meshes in the game - Melee and Ranged. Both were exported to the Blender modelling software, where the head was manually cut with the **Bisect** function and imported back to Unreal Engine as a Static Mesh. When the enemy is hit, every bone parented to the neck is hidden and on their place the severed Head Mesh is spawned with a physics impulse along the slice trajectory.

The code for this solution can be seen in Listing 35.

```
bool ABaseEnemy::HandleHitReaction(const FCombatHitData& CombatHitData)
{
  if (!bIsGettingHit)
  {
    ApplyDamage(CombatHitData); //call to blueprint

    GetMesh()->HideBoneByName(HeadBoneName, EPhysBodyOp::PBO_None);

    AStaticMeshActor* SpawnedHead = GetWorld()->SpawnActor<
    AStaticMeshActor>(AStaticMeshActor::StaticClass());
    SpawnedHead->SetMobility(EComponentMobility::Movable);

    SpawnedHead->SetActorTransform(GetMesh()->GetBoneTransform(
    HeadBoneName));

    auto Location = SpawnedHead->GetActorLocation();
    Location.Z += 20.f;
    SpawnedHead->SetActorLocation(Location);

    SpawnedHead->SetActorScale3D(FVector(1.4f));

    FVector HeadImpulse = (CombatHitData.CutPlaneNormal + CombatHitData.
    CutVelocity).GetSafeNormal() * DismembermentStrength;

    UStaticMeshComponent* MeshComponent = SpawnedHead->
    GetStaticMeshComponent();
    if (MeshComponent)
    {
      MeshComponent->SetStaticMesh(HeadMesh);
      MeshComponent->SetSimulatePhysics(true);
      MeshComponent->SetEnableGravity(true);
      MeshComponent->SetCollisionEnabled(ECollisionEnabled::PhysicsOnly);
      MeshComponent->AddImpulse(HeadImpulse);
      MeshComponent->SetAllMassScale(1);
    }

    bIsGettingHit = true;
    return false;
  }

  return true;
}
```

Listing 35: Enemy damage and head dismemberment (source: own elaboration)

6 Game Testing

Testing was important in our game developing process having an Agile work structure. [1] It was the time, that was chosen as deadlines for some parts of the game. The most important part was to have a build of the project without any errors for the day of game testing.

6.1 Regular Game Tests

For each game testing session, the authors have divided testers into two groups. The first group consists of players who have not had a chance to play the game yet, while the second group included individuals who provided ongoing feedback on newly developed parts of the game. The developers were aiming to have at least one player in the first group and at least two people in the second group.

We have tried to involve as many testers as possible to gather more feedback. Not only it was crucial for identifying different errors, that the authors may have overlooked, but also for observing how players respond to the new sections of the game. Even if the game is designed for a specific audience, it is still crucial to see how different players react to ensure the developers are heading in the right direction.

6.2 Getting Feedback

Whenever a new tester started playing the game, at least two developers were present to observe and assist. Each observer had their own role:

1. **The Observer**

   – **Primary Task**
   The Observer was responsible for taking detailed notes and gathering all possible data.
   – **Activities During Testing**
   - They quietly observed and documented the tester's actions, focusing on anything the tester did that was unintended from the game's design. - Special attention was paid when the tester

interacted with new mechanics. - the Observer listened carefully to any comments, sounds, or feedback the tester provided during the gameplay.

- **Activities After Testing**
  - They asked the tester a set of prepared main questions related to the design, gameplay and overall experience. They could also ask for specific thoughts about recently added mechanics or features.

2. **The Helper**

   - **Primary Task**
     - The Helper ensured that the testing process run smoothly by assisting the tester in navigating the game and addressing any unexpected issues that arise.
   - **Activities During Testing**
     - If the game encounters any bugs or unintended behaviour, the Helper helped guide the tester past the issue to continue the session. - They could try to engage in a light conversation with the tester to capture spontaneous opinions or impressions while they play.
   - **Activities After Testing**
     The Helper shared real-time insights with the Observer to enrich the understanding of the tester's experience.

7 Conclusions

The project is a product that involves skills from all computer science aspects, User Interface and User Experience, Algorithms, Vector Math, Programming patterns, Data collections, Artificial Intelligence and more. Yet, the developers have managed to achieve the experience of fps precision and player agility, the elements described below have helped to achieve this effect.

Player agility was achieved by Parkour system (section 5.1) that allows the players to traverse through the levels with the impressive pace thanks to comprehensive features such as wall running or sliding. With the mantle and crouch abilities even more more routes are now open to explore. The combination of those capabilities, along with the utility systems such as the Coyote Time, gives the players virtually endless possibilities to finish the level.

The game's levels (section 5.10) are focused on fast-paced parkour, strategic enemy placement and customized obstacles. Enemies and obstacles are positioned to challenge players while offering multiple paths to complete each level. An appealing art style elevates the player's experience. The progression starts with a tutorial that introduces the core mechanics to increasingly complex levels that balance accessibility with competitive gameplay.

Combat System (section 5.3) features a number of interesting and well-thought solutions to help the player feel like a powerful ninja. Its robust and quality-focused implementation allowed to amplify the impact of eliminating dozens of deadly adversaries in a satisfying way, without interfering with the level and enemy design.

The Combo System (section 5.4) additionally enhances the sense of mastery and precision by introducing points that are given to the player for performing custom actions or killing enemies within the time limit. This system encourages the player to defeat as much enemies as possible in a short time.

The Special Abilities (sections 5.6 and 5.3) feature provides two powerful mechanics: Time Stop and Super Ability. Time Stop allows the player to stop all enemies in place giving an opportunity to precisely deliver a finishing blow. On the other hand Super Ability freezes time, allowing the player to teleport up to 4 attackers killing them instantly.

Rewind Time mechanic (section 5.6) allows player to turn back time on death once per level, providing an additional opportunity for those with less precision or agility to recover from mistakes.

The enemy system (section 5.5) features advanced AI, supporting two enemy types: melee and ranged. Close-combat enemies rely on vision and hearing, while ranged enemies additionally leverage the Environment Query System for enhanced movement. heir abilities, such as the melee enemy's teleportation and the ranged enemy's shield, are perfectly matched with the parrying system. Both types make game challenging and incredibly satisfying as player navigates encounters with them.

User Interface (section 5.9) was created with the great care. It is minimalistic, yet streamlined to display only the information that players should receive without pointlessly obfuscating the view and overloading their attention with unnecessary details.

In the end, the project successfully illustrates a synergy of skills and technologies from across the field of computer science, completely aligning with its thesis as the authors desired.

Looking forward, the potential for further development lies in refining the gameplay mechanics to achieve a new niche yet highly engaging genre - Speedrun Stealth FPS. Not only will this direction emphasize an achieved blend of high-speed traversal and brutal combat, but it will also bring a new authentic and unique vision to the game. It will leverage precision-based stealth and tactical decision making. By making these enhancements, the project could evolve from its current state into an innovative experience.

References

1. Bryant, R.D.: Game testing all in one (2024), https://www.google.pl/books/edition/Game_Testing/nNYJEQAAQBAJ?hl=pl&gbpv=1&dq=game+testing+book&printsec=frontcover
2. Ghost of tsushima steam chart, https://steamdb.info/app/2215430/charts/
3. Ghostrunner chart, https://steamdb.info/app/1139900/charts/
4. Ghostrunner steam market reviews, https://store.steampowered.com/app/1139900/Ghostrunner/
5. Github, https://docs.github.com/
6. Johnston, P.: Creating a circular buffer in c and c++ (2017), https://embeddedartistry.com/blog/2017/05/17/creating-a-circular-buffer-in-c-and-c/
7. Kulon, B.: Parkour: How to improve freedom of movement in first-person games in 20 simple steps (2018), https://www.gdcvault.com/play/1025208/Parkour-How-to-Improve-Freedom
8. Mirrors edge chart, https://steamdb.info/app/1233570/charts/
9. Neon white chart, https://steamdb.info/app/1533420/charts/
10. Newman, A.: Unsynced: The last of us melee system (2014), https://gdcvault.com/play/1020368/Unsynced-The-Last-of-Us
11. Nystrom, R.: Game Programming Patterns. Genever Benning, 1st edn. (2014), https://gameprogrammingpatterns.com/
12. Options pattern in .net (2024), https://learn.microsoft.com/en-us/dotnet/core/extensions/options
13. Parberry, I., Dunn, F.: 3D Math Primer for Graphics and Game Development. TaylorFrancis Inc, 2nd edn. (2011), https://gamemath.com/
14. Perforce, https://www.perforce.com/p/vcs/free-version-control
15. Reid, M.: ue5-rewind: Time manipulation prototype (2024), https://github.com/NuMakesGames/ue5-rewind?tab=readme-ov-file
16. Rider, https://www.jetbrains.com/rider/
17. Rogers, P.: 41 video gaming statistics 2025 (users demographics) (2025), https://answeriq.com/video-gaming-statistics/
18. Rosen, D.: Animation bootcamp: An indie approach to procedural animation (2014), https://gdcvault.com/play/1020583/Animation-Bootcamp-An-Indie-Approach
19. Steam store charts, https://steamdb.info/charts/
20. Tondello, G.F., Nacke, L.E.: Player characteristics and video game preferences. In: Proceedings of the Annual Symposium on Computer-Human Interaction in Play. p. 365–378. CHI PLAY '19, Association for Computing Machinery, New York, NY, USA (2019). https://doi.org/10.1145/3311350.3347185, https://doi.org/10.1145/3311350.3347185
21. Zimmerman, C.: Master of the katana: Melee combat in ghost of tsushima (2021), https://gdcvault.com/play/1027194/Master-of-the-Katana-Melee
22. Zimmerman, C.: The Rules of Programming The Rules of Programming. O'Reilly Media, Inc. (2022), https://www.oreilly.com/library/view/the-rules-of/9781098133108/

List of Figures

Listings

8 Licenses

- Assets from Fab marketplace: https://www.fab.com/eula
- Katana Static Mesh: https://www.cgtrader.com/pages/terms-and-conditions#royalty-free-license
- Robot asset https://creativecommons.org/licenses/by/4.0/
- Sound effects: https://pixabay.com/service/license-summary/
- Music from WhiteBat Studio: https://whitebataudio.com/license-agreement/

Assets under MIT license:

- EpicLeaderboard Plugin - https://github.com/JR-UE4/EpicLeaderboard/tree/master (author: EpicLeaderboard.com)
- Advanced Sessions Plugin - https://github.com/mordentral/AdvancedSessionsPlugin (author: Joshua Statzer)
- Unreal Engine 5 Rewind - https://github.com/NuMakesGames/ue5-rewind (author: Marcus Reid)

MIT License